Master Thesis

Design and Evaluation of Efficient Semantic Company Retrieval at Scale

Marmee Pandya
(matriculation number 1963521)

September 15, 2026

Submitted to
Data and Web Science Group
Dr. Ralph Peeters
University of Mannheim

Abstract

A company can be highly relevant to a search query yet remain invisible to a keyword-based retrieval system when its description does not contain the query's exact terminology. This vocabulary mismatch motivates a central question for ISTARI.AI's company search product. Can embedding-based semantic retrieval, combined with approximate nearest-neighbour (ANN) search, generate accurate candidate sets efficiently at scale, and can it improve upon the existing keyword-driven production system? This thesis evaluates multiple dense embedding models, a late-interaction architecture, and several hybrid retrieval pipelines on a corpus of 98,716 companies. Under the initial production-derived evaluation protocol, a learned fusion ranker performs best, reaching Recall@1,000 of 0.823. However, this protocol treats a company as relevant only if the production system itself already retrieved it. Because a system cannot outperform the reference that defines its own evaluation labels, this circularity can obscure genuine improvements over production. To address this, the thesis builds an independent, multi-judge silver standard from LLM-based relevance assessments instead. These judgements are not ground truth, but because they are independent of production, they let production's own results be checked against a standard production did not define. Under this evaluation, production fails to retrieve 27.3% of the companies identified as relevant. On a separate pool of companies that production never ranked at all, the strongest single embedding model, Linq-Embed-Mistral, reaches Recall@1,000 of 88.5%. This shows that semantic retrieval's additional coverage extends beyond the companies production already surfaces. The independent evaluation also changes the interpretation of the fusion model's feature-importance results. Conclusions drawn under the production-derived protocol were not fully robust. The thesis further evaluates retrieval efficiency at increasing corpus sizes. At the largest tested scale of 397,025 companies, ANN search remains within 0.5–2.5 percentage points of exact search in Recall@1,000 and NDCG@1,000 (Normalised Discounted Cumulative Gain), while reducing query latency by 25–78× depending on corpus size. Statistical testing supports these differences at the tested scale, a favourable trend that would need to be confirmed at Istari's full production database. The results provide evidence that semantic retrieval can substantially improve candidate coverage over production, while approximate nearest-neighbour search maintains strong efficiency relative to exact search at the tested corpus sizes. More broadly, using production's own retrieval results as relevance labels can substantially constrain the conclusions a retrieval evaluation can draw. Building an independent standard need not mean exhaustive human annotation. This thesis's incrementally-expanded, LLM-judged silver standard shows that a small, carefully validated core can substitute for it when no human-labelled data exists.

Contents

Abstract ii

1 Introduction 1

2 ISTARI.AI and its Data 4
2.1 About ISTARI.AI and the Global Organisation Index 4
2.2 Data Pipeline and Methodology 4
2.2.1 Schema and Key Fields 4
2.2.2 Industry Classification: NACE Codes 5
2.2.3 Geographic Coverage 6
2.2.4 Evaluation Dataset 6

3 Literature Review 7
3.1 Sparse Retrieval 7
3.2 Dense Retrieval 8
3.3 Approximate Nearest Neighbour Search 9
3.4 Late Interaction and Retrieval-Specific Pretraining 9
3.5 Hybrid Retrieval, Reranking, and Learning to Rank 10
3.6 Company and Entity-Level Retrieval 10
3.7 Evaluation Without Human Relevance Judgements 11

4 Methodology 13
4.1 Phase I: Baselines 14
4.2 Phase II: Improving Retrieval 18
4.2.1 Extended Baseline Comparison 18
4.2.2 HyDE and RRF 19
4.2.3 ANN Index Scaling to 2 Million Vectors 20
4.2.4 Corpus Quality: Platform Boilerplate 20
4.2.5 Alternative Retrieval Architectures: SimLM and ColBERT 21
4.2.6 Full-Pool Reranking and Learned Fusion Ranker 22
4.3 Phase III: Evaluation Validity 23
4.3.1 Building the Silver Standard 23
4.3.2 Retraining and Expanding the Silver Standard 25
4.3.3 A Production-Independent Test 29
4.3.4 Silver-Standard Coverage Across the Ranking 30
4.4 Phase IV: Learned Retrieval Revisited 31
4.4.1 Hyperparameter Tuning and Neural Network Fusion Ranker 31
4.4.2 Fine-Tuning the Reranker 32

4.5 Phase V: Scalability . . . 33
4.5.1 Scaling to a Larger Corpus . . . 33
4.5.2 Approximate Nearest Neighbour (ANN) Index Tuning . . . 35

5 Results and Discussion **36**
5.1 Phase I: Baselines . . . 36
5.2 Phase II: Improving Retrieval . . . 38
5.2.1 Extended Baseline Comparison . . . 38
5.2.2 HyDE and RRF . . . 39
5.2.3 ANN Index Scaling and Recall@1,000 Error Analysis . . . 40
5.2.4 Corpus Quality . . . 41
5.2.5 Oracle Union Recall Ceiling . . . 42
5.2.6 Alternative Retrieval Architectures: SimLM and ColBERT . . . 42
5.2.7 Full-Pool Reranking and Reranker Ablation . . . 43
5.2.8 Learned Fusion Ranker . . . 45
5.3 Phase III: Evaluation Validity . . . 45
5.3.1 Building the Silver Standard . . . 45
5.3.2 Retraining and Expanding the Silver Standard . . . 46
5.3.3 Production Divergence and the Independent Ranker . . . 49
5.3.4 A Production-Independent Test . . . 52
5.3.5 Calibration Check and Synthesis of Evaluation Findings . . . 54
5.3.6 Silver-Standard Coverage and External Review . . . 59
5.4 Phase IV: Learned Retrieval Revisited . . . 61
5.4.1 Hyperparameter Tuning and Neural Network Fusion Ranker . . . 61
5.4.2 Fine-Tuning the Reranker . . . 63
5.5 Phase V: Scalability . . . 65
5.5.1 Scaling to a Larger Corpus . . . 65
5.5.2 ANN Index Tuning . . . 65
5.6 Overall Experimental Findings . . . 67

6 Conclusion **69**

7 Future Work **71**

Bibliography **73**

Code Availability **78**

Ehrenwörtliche Erklärung **79**

1 Introduction

A company that is a strong match for a search query can still be invisible to a keyword-based search system, simply because its own description uses different words than the query does. A search for a *"renewable energy consultancy"*, for example, will not surface a company that describes itself as a *"sustainability advisory service"*, even though the company may be exactly the kind of business the query is looking for. This is a well-known limitation of classical keyword-based retrieval. Sales teams targeting prospects, investors screening deals, and analysts mapping a market or a supply chain all depend on locating the right handful of relevant organisations out of a much larger pool, often millions of candidates. Providers such as ISTARI.AI build company intelligence platforms specifically to support this need, maintaining datasets containing tens of millions of organisation profiles, covering everything from small local businesses to large multinational enterprises. Existing workflows at these platforms, however, typically rely on exactly the manually configured keyword filters that are vulnerable to the mismatch problem described above.

This mismatch has a cost beyond any single missed result. An analyst who reformulates a query several times to work around unfamiliar terminology spends time that scales with how often the gap recurs, and a keyword-based system provides no signal about what it failed to retrieve in the process, so there is no direct way to know how many genuinely relevant companies were missed along the way.

At the same time, any system intended to operate on a company database of this size must satisfy strict constraints on efficiency and scalability. Retrieval has to remain accurate over a dataset that is high-cardinality in a specific sense. It spans tens of millions of distinct organisation profiles, any one of which could be the correct answer to a given query, rather than a small, fixed set of categories a classifier could simply learn to distinguish. It also has to return results quickly enough to support an interactive search experience, not just a slow, offline batch process.

One way to sidestep the vocabulary mismatch problem entirely would be to have a large language model directly judge every company profile against every query, since LLMs are demonstrably capable of recognising whether a description matches an intent even when the wording differs. This is not viable as a first-pass retrieval mechanism at Istari's scale. Even a fraction of a cent per company, multiplied across tens of millions of profiles for every incoming query, is prohibitively expensive, well before accounting for the latency an interactive product requires. Some cheaper mechanism has to narrow the full company database down to a manageable candidate set first, before any more expensive downstream stage, whether a human analyst or a further LLM pass, examines it. This thesis is about that first, cheaper stage. The goal is efficient, accurate candidate generation, not the full search experience end to end. Because the goal is a strong

shortlist rather than a final ranking, coverage matters more than precise ordering within it, which is why Istari prioritised Recall@1,000 as this thesis's primary metric throughout.

This thesis is conducted in collaboration with ISTARI.AI (Chapter 2 introduces the company and its data), working directly with Istari's production system and its actual company database. Consequently, the results reported here reflect two specific characteristics of this setting. Company profiles combine heterogeneous structured fields with AI-generated free text, described in full in Chapter 2, a data format that differs from the passage-retrieval benchmarks the wider literature is built around. And no independently constructed relevance judgements exist for this task, an evaluation constraint a benchmark-based study would not encounter, since benchmarks typically ship with human relevance labels already attached. Both characteristics shape this thesis's findings directly, as the rest of this chapter sets out.

Dense, embedding-based retrieval and approximate nearest-neighbour search are well established individually. Bi-encoder architectures, late-interaction models, and hybrid fusion methods have all been shown to outperform keyword matching on standard information retrieval (IR) benchmarks (Karpukhin et al. 2020) (Chapter 3 reviews this literature in full). What is missing is evidence for how these methods behave on large-scale company profiles specifically, an applied setting closer to entity-level retrieval and company-embedding work (Gerling 2023) than to general-purpose passage benchmarks, where records combine heterogeneous structured fields with AI-generated free text. This is the first gap this thesis addresses by applying established retrieval methods to a domain with genuinely different data characteristics.

A second, deeper gap concerns evaluation itself. Existing retrieval research is predominantly validated against benchmarks with human relevance judgements. For Istari's company-search task, no independently constructed relevance judgements were available for the full corpus and query set at the start of this project, since Istari's 20-million-company database has never been exhaustively labelled by anyone. In their absence, a natural fallback is to treat production's own retrieved companies as a proxy for relevance, a practical benchmark when human judgements are unavailable (Voorhees 2002; Thakur et al. 2021). Such a proxy is circular by construction, however it can support operational comparisons between systems, but it cannot establish whether a competing system retrieves relevant companies that production itself misses. Production's own output is also not a purely algorithmic ranking in the first place. According to Istari, search behaviour is manually configured on a per-customer basis to meet specific customer requirements, a further reason it cannot serve as an objective, consistent standard of relevance. This thesis treats that circularity as part of the research problem rather than a limitation to note in passing, and constructs an independent relevance standard, built incrementally rather than all at once (Section 4.3.2), to test it directly.

Problem statement. This thesis addresses two connected problems. First, it is not known whether embedding-based semantic retrieval can outperform Istari's existing keyword-based production system at production-relevant scale and query latency. Second, and more fundamentally, it is not even clear whether that comparison can be

measured validly at all, since the only available yardstick for what counts as relevant is production's own output.

Research questions. This problem translates into three research questions, addressed in turn across Chapters 4 and 5:

RQ1 Can embedding-based semantic retrieval, combined with approximate nearest-neighbour search, generate accurate top-k candidate company sets from a large-scale dataset while keeping query latency low, and how does its retrieval quality compare to Istari's keyword-based production system?

RQ2 Does combining multiple, complementary retrieval signals, culminating in a model trained directly on this task's own queries, retrieve more relevant companies than any single retrieval method alone?

RQ3 Is the production-derived evaluation protocol used to answer RQ1 and RQ2 itself trustworthy, and do the retrieval gains observed under it still hold once tested against an independently constructed, non-circular relevance standard?

Scope. This thesis comes with explicit boundaries. "At scale" throughout this thesis means the largest corpus directly evaluated. This corpus reaches nested tiers of up to 397,025 companies (Section 4.5.1), roughly 2% of Istari's full 20-million-company database. It does not mean an end-to-end evaluation at full production scale, a ceiling driven as much by practical limits as by design. Provisioning a dataset the size of Istari's entire proprietary database was itself difficult to arrange, and encoding and judging it would have required far more compute and a much larger LLM-judging budget than this thesis could accommodate. The experiments instead establish a scalability trend up to the tested size, and Chapter 7 discusses what would be needed to extend it further. This thesis evaluates retrieval quality and computational efficiency under the query set and corpus described in Chapter 4. It does not evaluate the end-user search experience, run a live production A/B test, or build and deploy a production system. Every result reported here is measured offline against the datasets and pipelines described in that chapter.

Thesis outline. The remainder of this thesis is organised as follows:

- Chapter 2 introduces Istari's Global Organisation Index, the dataset underlying this thesis.
- Chapter 3 reviews the relevant retrieval and evaluation literature.
- Chapter 4 describes the experimental setup and the design of every baseline and experiment.
- Chapter 5 reports the results, findings, and interpretation for each of those experiments.
- Chapter 6 summarises the main findings against this thesis's original research questions and Chapter 7 outlines concrete directions for future work.

2 ISTARI.AI and its Data

2.1 About ISTARI.AI and the Global Organisation Index

This thesis is conducted in collaboration with ISTARI.AI[1], a data intelligence company specialising in firmographic data at scale. Istari's core product is a curated, organisation-level dataset covering organisations worldwide, enabling use cases such as market analysis, company search, and lead generation.

The primary dataset used in this thesis is Istari's **Global Organisation Index** (GOI), a curated collection of approximately 20 million active organisations spanning 232 countries and territories. The GOI is the canonical source of firmographic data within Istari's data infrastructure, covering organisation identity, location, industry classification, organisation size, and other firmographic attributes.

2.2 Data Pipeline and Methodology

The GOI is constructed through a multi-stage validation pipeline that deliberately prioritises data quality over volume. National and open registers are queried worldwide, yielding an initial pool of approximately 400 million organisations. Each is then assessed for attribution to an identifiable web domain, reducing the pool to approximately 40 million; the remainder are excluded as inactive, never truly operational, or lacking a web presence. A final verification step confirms that the domain-attributed organisations are still actively operated, reducing the dataset to the approximately 20 million organisations that make up the GOI, every one of which is therefore both domain-verified and confirmed active.

2.2.1 Schema and Key Fields

Each record in the GOI contains a set of core identity fields, geographic fields, and classification fields. The fields most relevant to this thesis are summarised in Table 2.1.

Of particular importance for this thesis is the `summary` field. This field contains an AI-generated natural language description of each organisation, synthesised from its web presence and available structured data. It is the most semantically rich and consistently populated field in the dataset, with effectively no missing values in the experimental corpus, and therefore anchors the text representation fed to every retrieval model evaluated in this thesis, alongside the other GOI fields. This field is not without its own data-quality issues, however. For a subset of query topics, the summary instead

[1] https://docs.istari.ai/goi/goi_start_page/

Table 2.1: Key fields of the GOI schema used in this thesis.

Field	Type	Description
`name`	STRING	Organisation name
`domain`	STRING	Organisation website domain (unique identifier)
`summary`	STRING	AI-generated textual summary of the organisation
`keywords`	LIST	Keywords
`employee_class`	STRING	Employee count bracket
`country / country_code`	STRING	Country and ISO code
(subdivisions)	STRING	`state`, `region`, `district`, `municipality` (and their codes), at decreasing levels of granularity
(address)	STRING or FLOAT64	`address`, `latitude`, `longitude`
`nace_code`	STRING	NACE industry classification code
`organization_type / _size`	STRING	Type (Company, Startup, Academic, Public, Other) and size (Micro, Small, Medium-sized, Large enterprise)

describes a website hosting or platform provider rather than the company's own business, an issue quantified directly in Section 5.2.4.

2.2.2 Industry Classification: NACE Codes

Industries in the GOI are classified using a taxonomy Istari refers to as **NACE**[2] (*Nomenclature statistique des activités économiques dans la Communauté européenne*), the European Union's standard statistical classification of economic activities. Table 2.2 lists the five most represented sections in the dataset.

Table 2.2: Top 5 industries in the GOI by organisation count.

NACE	Industry	Count	Share (%)
N	Professional, scientific & technical activities	3,093,879	18.03
G	Wholesale & retail trade	2,240,191	13.05
C	Manufacturing	2,006,435	11.69
R	Human health & social work	1,440,340	8.39
K	Telecom, IT & computing	1,269,131	7.39

Organisation size in the GOI is derived from employee and revenue signals and bucketed into four categories (Micro, Small, Medium-sized, Large enterprise) following the

[2] https://ec.europa.eu/eurostat/web/nace

European Union's definitions of small and medium-sized enterprises (SME); the distribution is heavily skewed towards smaller organisations, with over 86% of the GOI classified as micro or small. This field is not used as a retrieval feature in the experiments that follow.

2.2.3 Geographic Coverage

The GOI covers 232 countries and territories. Table 2.3 lists the top three countries by volume.

Table 2.3: Top 3 countries by organisation count in the GOI.

Country	**Organisations**
United States	3,626,794
Germany	1,828,181
United Kingdom	1,013,026

Approximately 2.7 million organisations in the GOI do not have a standardised administrative region. These records are intentionally retained as they remain valuable for non-geographic analyses such as industry or domain-level retrieval, the primary focus of this thesis.

2.2.4 Evaluation Dataset

Istari provided an evaluation dataset consisting of 101 real-world search queries along with each query's top-1 000 production results, yielding 101 000 (query, company) pairs. Since the same company can appear across multiple queries' top-1 000 results, this thesis deduplicates the pairs by `domain`, keeping a single record per company: **98 716 unique companies** remain, and all retrieval experiments in Chapter 4 are conducted over this corpus. This corpus should not be interpreted as a random sample of the full GOI; it is derived from the companies production itself returned for the 101 queries (Section 4.1 defines "relevant" precisely).

3 Literature Review

This chapter reviews the retrieval paradigms, applied literature, and evaluation practices this thesis builds on and compares against, organised as follows:

- Section 3.1 covers sparse retrieval and Section 3.2 covers dense retrieval, the two foundational paradigms for the baselines described in Chapter 4.
- Section 3.3 covers approximate nearest neighbour search, the technique that makes dense retrieval practical at scale.
- Section 3.4 covers late-interaction retrieval and retrieval-specific pretraining, the two alternative paradigms evaluated in Section 4.2.5.
- Section 3.5 covers hybrid retrieval, cross-encoder reranking, and learning to rank, the techniques underlying the hybrid pipelines in Section 4.2.6.
- Section 3.6 reviews the applied literature closest to this thesis's own task, entity matching, entity resolution, and company-level embeddings, and identifies what that literature does not yet address.
- Section 3.7 covers evaluation practice in information retrieval when human relevance judgements are unavailable, directly motivating the pseudo-relevance protocol used throughout this thesis (Section 4.1).

3.1 Sparse Retrieval

Text retrieval systems fall into two broad families. **Sparse** retrieval represents queries and documents as high-dimensional vectors of term weights and scores matches by term overlap. This tradition traces back to the vector space model, where each document is represented by weighted term frequencies rather than raw counts (Salton and Buckley 1988), establishing term weighting itself, rather than exact phrase matching, as the basis for ranking. A widely used sparse retrieval method built on this foundation is BM25 (Robertson and Zaragoza 2009), a probabilistic scoring function that weighs each query term by its frequency in a document and its rarity across the whole collection. BM25 is fast, requires no training, and remains a strong baseline decades after its introduction. However, it cannot reliably match queries and documents that describe the same concept using different words, a limitation known as the vocabulary mismatch problem and directly observed for this thesis's task in Section 5.1.

More recent work keeps sparse retrieval's efficient inverted-index infrastructure while using neural networks to address this weakness directly. Doc2query (Nogueira et al. 2019) trains a sequence-to-sequence model to predict queries a document is likely to answer, then appends those predicted queries to the document before indexing, so a document can match a query term it never originally contained. SPLADE (Formal

et al. 2021) goes further, learning sparse term-weight expansions for both documents and queries directly via a masked-language-model head, rather than relying on literal term frequency alone, while still producing a sparse representation compatible with a standard inverted index. Both approaches sit between the sparse and dense families this section and the next describe separately, preserving sparse retrieval's exact-match efficiency while borrowing some of the vocabulary generalisation that motivates dense retrieval. Neither is evaluated directly in this thesis, whose lexical baseline is plain BM25 (Section 4.1), but they mark a relevant middle ground that this thesis's own hybrid and fusion experiments (Section 4.2.2 onward) address empirically instead, by combining separate sparse and dense signals rather than learning a single expanded sparse representation.

3.2 Dense Retrieval

Dense retrieval instead represents queries and documents as fixed-size vectors learned by a neural encoder, and scores matches by vector similarity rather than literal term overlap. Dense Passage Retrieval (DPR) (Karpukhin et al. 2020) popularised the bi-encoder architecture for open-domain retrieval, using separate encoders for queries and documents (or, in this thesis, companies). The two encoders are trained so that relevant query-document pairs are close in vector space, while each side is encoded independently. This allows documents to be embedded once, offline, and their representations reused across future queries. Sentence-BERT (Reimers and Gurevych 2019) showed that the same bi-encoder idea, applied to general-purpose sentence embeddings rather than a retrieval-specific training objective, produces embeddings useful for semantic similarity and retrieval tasks broadly, and underlies the general-purpose sentence-transformer models (including MiniLM, Section 4.1) used throughout this thesis.

More recent embedding models, such as the BGE family (Xiao et al. 2024; Chen et al. 2024), combine this bi-encoder architecture with large-scale contrastive pretraining and hard-negative mining, and have become strong general-purpose dense retrieval models, forming the basis of several baselines evaluated in Sections 4.1 and 4.2.1.

Two distinct further directions in embedding model development are also represented among this thesis's extended baselines (Section 4.2.1): GTE (Li et al. 2023) and Nomic Embed (Nussbaum et al. 2024) show that a comparatively small (100–300M-parameter) encoder, trained with careful multi-stage contrastive learning over diverse data, can match or exceed much larger models. E5-Mistral (Wang et al. 2024), SFR-Embedding-Mistral (Meng et al. 2024), and Linq-Embed-Mistral (Linq AI Research 2024) instead take the opposite approach, adapting an already-large 7-billion-parameter instruction-tuned language model (Mistral-7B) into an embedding model via further contrastive fine-tuning. This split between "small and carefully trained" and "large and repurposed" is directly tested for this thesis's own task in Section 5.2.1, where it does not resolve in the direction model scale alone would predict.

Public leaderboards such as the Massive Text Embedding Benchmark (MTEB) (Muennighoff et al. 2023) rank models like these against one another across many tasks, but

its results, like all such leaderboards, reflect performance averaged over its constituent benchmark datasets, not any particular downstream domain. The extent to which a strong MTEB ranking transfers to company-profile retrieval specifically is itself part of what Section 5.2.1 investigates.

3.3 Approximate Nearest Neighbour Search

Dense retrieval requires finding the nearest vectors to a query vector among the full document collection. Exact search costs time linear in the number of documents, which becomes impractical at the scale of millions of documents relevant to this thesis's target deployment (Section 2). Approximate nearest neighbour (ANN) search instead searches only a carefully chosen subset of the collection, avoiding a full linear scan. The central trade-off is between recall, how often the true nearest neighbours are actually found, and query latency, and unlike exact search this trade-off is tunable. It depends on the index structure and its search-time parameters, not on a single fixed accuracy cost paid by approximating at all. FAISS (Johnson et al. 2019) provides GPU-accelerated implementations of both exact and approximate search, including inverted-file indexes (IVF), which cluster the embedding space in advance and, at query time, search only the clusters nearest the query rather than every vector, reducing candidate comparisons at the cost of possibly missing a true neighbour that fell into a cluster the query never visits. Hierarchical Navigable Small World graphs (HNSW) (Malkov and Yashunin 2020) take a different approach. They build a multi-layer proximity graph over the document vectors at index-construction time, then greedily traverse it at query time, so the accuracy-latency trade-off is governed by how deep and wide that traversal is allowed to go rather than by how many clusters are searched. Both approaches also carry a memory and index-build cost that a linear scan does not, IVF's cluster centroids and HNSW's proximity graph must be built once, in advance, and held in memory alongside the vectors themselves, a cost this thesis's own scaling experiment tracks directly rather than assumes away. Both IVF and HNSW are evaluated directly in this thesis's ANN scaling experiment (Section 4.2.3), which tests whether their recall-latency trade-off remains favourable as corpus size grows towards simulated production scale.

3.4 Late Interaction and Retrieval-Specific Pretraining

A bi-encoder represents an entire document with a single fixed-size vector, which can discard fine-grained information relevant to a specific query. ColBERT (Khattab and Zaharia 2020) addresses this with *late interaction*. Queries and documents are each represented as sets of per-token vectors, and relevance is scored by summing, for each query token, the maximum similarity to any document token (MaxSim). This avoids requiring a single vector to summarise the whole document in advance.

This adds compute and storage cost relative to a single-vector bi-encoder, since retrieval now requires comparing token sets rather than single vectors. ColBERTv2 (Santhanam et al. 2022b) introduces residual compression of the per-token vectors, sub-

stantially reducing this cost, and the subsequent PLAID (Performance-optimized Late Interaction Driver) engine (Santhanam et al. 2022a) builds on that compressed representation to make late-interaction search efficient at scale. This thesis instead implements MaxSim scoring directly (Section 4.2.5), trading PLAID's efficiency for an exact, dependency-light implementation suited to the scale of the evaluation corpus used here.

A separate line of work improves dense retrieval not through architecture but through pretraining. SimLM (Wang et al. 2022) pretrains a bi-encoder with a representation-bottleneck objective before any retrieval-specific fine-tuning, explicitly designed to produce embeddings well suited to downstream passage retrieval. This thesis evaluates whether that retrieval-specific pretraining transfers to a domain (company profiles) quite different from the passage-retrieval benchmarks it was designed for (Section 4.2.5).

3.5 Hybrid Retrieval, Reranking, and Learning to Rank

Sparse and dense retrieval, and different dense models, make different errors. A hybrid system that combines multiple retrieval signals can recover from a single method's individual weaknesses. Reciprocal Rank Fusion (RRF) (Cormack et al. 2009) is a simple, training-free method for combining several ranked lists. Each document's fused score is computed as the sum of $1/(k_{\mathrm{RRF}} + \mathrm{rank})$ across the lists in which it appears, requiring only that documents be ranked rather than that their raw scores be comparable across methods. RRF underlies the hybrid baseline in Section 4.2.2.

A complementary technique is *reranking*. A first-stage retriever returns a candidate set cheaply, and a more expensive model reorders it. Cross-encoder rerankers, such as the `bge-reranker-v2-m3` (BAAI 2024) and `mxbai-rerank-large-v1` (Mixedbread AI 2024) models used in Section 4.2.6, feed the query and each candidate into the same transformer jointly, rather than encoding them independently as a bi-encoder does. This lets the model attend directly between query and candidate tokens, typically producing more accurate relevance scores than a bi-encoder (Nogueira and Cho 2019), at the cost of requiring a full forward pass per candidate rather than a single fast vector comparison, so cross-encoders are used to rerank a small candidate set rather than to search a full collection.

More generally, combining multiple retrieval signals into a single relevance score can be framed as *learning to rank* (Liu 2009). Instead of a fixed, hand-designed combination rule such as RRF, a model is trained directly on labelled query-document pairs to learn how to weight and combine whatever features are available (similarity scores, ranks, or other signals) into a final ranking. This is the approach taken by the learned fusion ranker in Section 4.2.6, implemented with the logistic regression and gradient-boosted tree models provided by scikit-learn (Pedregosa et al. 2011).

3.6 Company and Entity-Level Retrieval

Much of the retrieval literature reviewed above is developed and evaluated on general-purpose passage and document retrieval benchmarks, questions answered from Wikipedia

or news text. Comparatively little of it addresses retrieval over company-profile data specifically. The applied literature closest to this thesis's own task instead comes from *entity matching* and *entity resolution*. Both address the problem of deciding whether two records, often company or product records drawn from different sources, refer to the same real-world entity. DeepMatcher (Mudgal et al. 2018) was among the first to systematically apply deep learning to this problem, and Ditto (Li et al. 2020) later showed that fine-tuning a pretrained language model on the same task, framed as a sequence-pair classification problem, achieves very high accuracy, over 96% F1 on company-matching benchmarks specifically. More recently, Peeters et al. (2025) show that large language models, prompted directly rather than fine-tuned for the task, can match or exceed these earlier approaches on the same entity matching benchmarks. Entity matching and the retrieval task addressed in this thesis are related but distinct. Matching decides whether two already-identified records are the same entity, a pairwise classification problem, while retrieval must first find, from a natural-language query, which of millions of candidate companies are even worth considering, a search problem over an entire corpus rather than a single pair.

Closer still to this thesis's own setting is Company2Vec (Gerling 2023), which builds company embeddings directly from the unstructured text of corporate websites, the same kind of web-derived company description this thesis's own AI-generated summaries are built from, and uses those embeddings to identify similar companies for peer-firm and industry-classification tasks. Company2Vec demonstrates that web-derived company text carries a usable semantic signal for retrieval-adjacent tasks, but it does not address ranked retrieval against a natural-language query, evaluate against a validated relevance standard, or test at the scale addressed in this thesis. This is precisely the gap this thesis fills. General-purpose semantic retrieval is well studied, and company-level embeddings have been shown to work for adjacent tasks like matching and peer-firm identification. Within the literature reviewed for this thesis, no study was identified that evaluates full retrieval, from a natural-language query to a ranked set of company candidates, on data and at a scale comparable to this thesis's own, under an evaluation protocol whose own validity has been separately established.

3.7 Evaluation Without Human Relevance Judgements

Even once a retrieval task and domain are defined, evaluating a system against it requires a way to decide which results are actually relevant. Standard information retrieval evaluation compares each system's ranking against a set of human relevance judgements for a fixed query set, historically constructed through pooling. In pooling, the top results from several systems are merged and judged by human assessors, as established by the methodology of the Text REtrieval Conference (TREC) (Voorhees 2002). No such human-annotated relevance judgements exist for the company search task addressed in this thesis. In their absence, this thesis instead adopts a *pseudo-relevance* protocol (Section 4.1). Under this protocol, a company is treated as relevant if it appears in the existing production system's own results for that query. This provides a practical way to

compare systems when human judgements are unavailable, although the resulting scores measure agreement with the reference system rather than absolute relevance, a limitation this thesis states explicitly wherever pseudo-relevance labels are used. Separately, the BEIR benchmark (Thakur et al. 2021) demonstrates the value of evaluating retrieval methods across diverse tasks and domains using established relevance assessments, rather than relying on a single retrieval task or domain.

This distinction matters because it was not obvious from the outset. Absent any human-verified relevance data, this thesis's own early experiments (Phases I–II, Section 4.1 onward) worked with production's top results as a practical stand-in for correctness, since no alternative existed at the time. Phase III (Section 4.3) is where this assumption was examined directly rather than continuing to rely on it by default, and found to be circular: a system cannot be shown to outperform production if production's own output is what defines correctness. Building a genuinely independent relevance standard from scratch, rather than continuing to lean on production's output, ran into a second, practical constraint. Istari's Global Organisation Index covers roughly 20 million companies, so exhaustively human-labelling relevant companies for even a modest query set was not feasible within this thesis's timeframe or budget (Section 4.3.2 quantifies this directly). This is why Phase III instead builds a small, independently judged core set first and expands it incrementally using LLM judges and active learning, rather than attempting full human annotation at either scale.

A more recent alternative to human pooling, and the one this thesis relies on for its independent silver standard (Section 4.3.1), is to use large language models themselves as relevance judges. Thomas et al. (2024) show that LLM judgements can be useful for large-scale system comparison, but agreement with human judgement, prediction of user preference, and agreement between LLM judges are related, distinct claims, and reliability on any of them depends on the task, the prompting procedure, and the judge model used. Published findings on one task or judge pairing therefore cannot simply be assumed to transfer to this thesis's own. This is exactly why this thesis's own blind calibration check (Section 5.3.1) tests agreement directly against its own judges and task, rather than relying on the literature's findings alone. This approach is not without open problems. A broader survey of LLM-as-a-judge methods (Li et al. 2024) documents recurring failure modes across the wider literature, including inconsistency across repeated runs, sensitivity to the order candidates are presented in, and systematic scoring biases that do not necessarily show up in simple percent-agreement figures, part of the motivation for this thesis's use of Cohen's kappa rather than raw agreement alone (Section 5.3.1) and for cross-checking judges from different model families rather than relying on a single model's judgements throughout. Because these labels are produced through a model-assisted, human-adjudicated process rather than exhaustive expert annotation, this thesis refers to them throughout as a **silver standard** rather than a gold standard, a distinction discussed further in Section 4.3.1.

4 Methodology

This chapter describes the experimental setup, evaluation protocol, computational environment, and the design of every retrieval baseline and experiment carried out for the company search task over Istari's Global Organisation Index (GOI). It answers *what was built and why*. The companion chapter, Chapter 5, reports *what happened after the experiments ran*.

This chapter and Chapter 5 share the same five conceptual phases. A phase's method is described here, and its results are reported under the same phase number in Chapter 5.

- *Phase I, baselines* (Section 4.1, results in Section 5.1): lexical and dense retrieval compared under the production-derived pseudo-relevance protocol.
- *Phase II, improving retrieval* (Section 4.2, results in Section 5.2): query expansion, hybrid fusion, reranking, and additional retrieval paradigms, all still evaluated against production-derived labels, culminating in the learned fusion ranker.
- *Phase III, evaluation validity* (Section 4.3, results in Section 5.3): the discovery that Phase II's labels are circular, and the construction, validation, and active-learning expansion of an independent silver standard for subsequent evaluation.
- *Phase IV, learned retrieval revisited* (Section 4.4, results in Section 5.4): tuning and fine-tuning experiments re-run against the new, non-circular standard, with paired significance testing added for the comparisons that matter most.
- *Phase V, scalability* (Section 4.5, results in Section 5.5): the pipeline tested on a corpus roughly four times larger, with IVF and HNSW approximate search re-evaluated at that scale to check whether their recall-latency trade-off still holds.

Each phase's central question is listed at the start of Chapter 5; each phase's evaluation protocol is given in Table 4.1 below.

The pseudo-relevance protocol defined in Section 4.1 (production's top-1 000 results per query) is used throughout this chapter's baseline, reranking, fusion, and oracle-ceiling experiments (Sections 4.1–4.2.4). This thesis later introduces two further, independent notions of relevance, once the circularity of trusting production's own output is identified and addressed (Section 4.3.1 onward). Table 4.1 previews all three so that later references to "the silver standard" or "the production-independent set" can be looked up in one place, even though the latter two are not defined in full methodological detail until the sections indicated.

The silver standard should be read with caution, not treated as ground truth. The silver standard is not a gold standard, and every result evaluated against it in this thesis should be read accordingly. It rests on LLM judgements with only minimal, and not fully independent, human checking. The author resolves judge disagreements (roughly

Table 4.1: The three relevance protocols used across this thesis, in order of introduction.

Protocol	**A company counts as relevant if. . .**	**Used for**
Pseudo-relevance	it appears in production's top-1,000 for that query	Introduced in Section 4.1. Used for baseline comparison, reranking, hybrid fusion, and oracle-ceiling experiments (Phases I–II).
Silver standard	two independent LLM judges agree (or a human resolves their disagreement) that it is relevant, drawn from a pooled, multi-source candidate set	Introduced in Section 4.3.1. Used for fusion-ranker retraining, tuning, fine-tuning, and all comparisons in Phase III–IV.
Production-independent	the same silver-standard judging process is applied to a candidate pool drawn exclusively from companies production never ranked for that query	Introduced in Section 4.3.3. The one test in this thesis not subject to production-pooling bias by construction.

16% of candidates pooled across every judging round, Figure 4.1), and those same resolved labels then score the author's own retrieval methods, a conflict of interest that is disclosed and partially mitigated but not eliminated (Section 5.3.5 discusses this in full). The candidates it covers were also not drawn randomly. Active-learning sampling deliberately targets ambiguous, boundary-case candidates for labelling (Section 4.3.2), so the standard is systematically skewed towards harder cases rather than a representative sample of all relevant companies. For these reasons, the silver standard is best understood as a considerably more rigorous alternative to trusting production's own circular output, not as an infallible reference. Every Recall and NDCG figure computed against it in this thesis should be read as an estimate with these specific, disclosed sources of uncertainty, not as an exact measurement.

Only the last two protocols are independent of production's own output; pseudo-relevance, used throughout Phases I–II of the roadmap above, measures agreement with production rather than an externally verified notion of relevance, and results reported under it should be read with that limitation in mind (Section 4.1 states this explicitly).

4.1 Phase I: Baselines

Dataset and Evaluation Protocol. The evaluation corpus is the 98 716-company set described in Section 2.2.4, built from the top-1 000 production results for 101 queries supplied by Istari. Those queries span a wide range of company search intents, including industry-specific queries (e.g. *"pharmaceutical companies in the US"*), geographic queries (e.g. *"tech startups in Berlin"*), and functional queries (e.g. *"B2B SaaS companies"*).

Every retrieval model in this thesis is built primarily around the `summary` field (Sec-

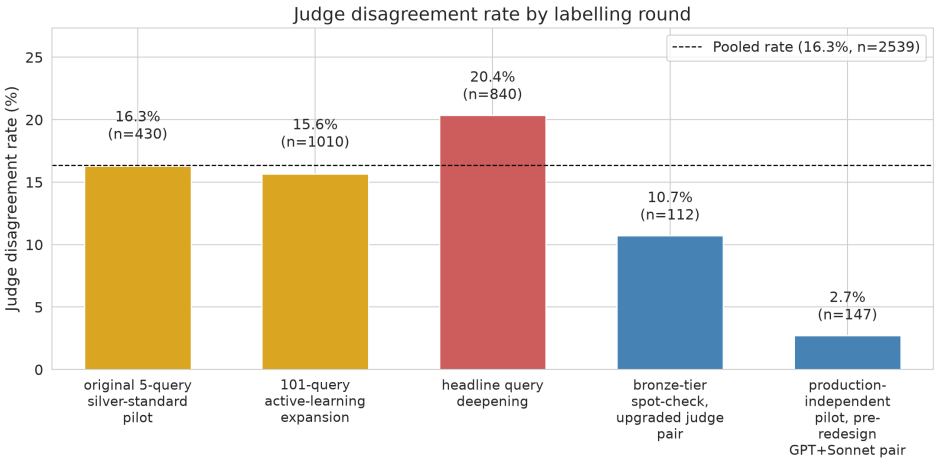


Figure 4.1: Judge disagreement rate by labelling round. The dashed line marks the pooled rate across all 2,539 doubly-judged candidates.

tion 2.2.1), the only GOI attribute detailed enough in free text to support genuine semantic matching against queries like those above; structured fields such as industry codes or founding year simply carry too little text to match against. That reliance is not free of cost. Section 5.2.4 quantifies how often the field instead describes a website host or platform provider rather than the company itself, and this limitation is carried through the results that follow rather than corrected away.

Pseudo-relevance labels. No human-annotated relevance judgements are available for the Istari company search task. Following standard practice in information retrieval research when ground-truth labels are absent (Voorhees 2002), this thesis adopts a *pseudo-relevance* assumption. A company is considered **relevant** for a given query if and only if it appears in the **top-1 000 results** of Istari's production search system for that query. This yields a relevance set of 1 000 companies per query, representing ∼1% of the total corpus. This thesis deliberately chooses this threshold over a stricter one so that reranking, fusion, and oracle-ceiling experiments later in this chapter have enough relevant companies per query to produce meaningful comparisons over large candidate pools.

This assumption has an important implication for the evaluation. The metrics measure the degree to which each retrieval system agrees with the production system, not whether it recovers objectively relevant companies. Consequently, a retrieval system that is strictly better than the production system may appear to underperform because it surfaces companies the production system missed. This limitation is acknowledged throughout the analysis.

Metrics. Three standard information retrieval metrics are reported at cut-offs $k \in \{10, 50, 100, 1000\}$:

Writing R_q for the set of relevant companies for query q and $T_k(q)$ for the top-k

companies a system retrieves for q:

- **Precision@k**: the fraction of the top-k retrieved companies that are relevant, Precision@$k = \frac{|T_k(q) \cap R_q|}{k}$. This measures user-facing quality, what a user sees on the first page of results.
- **Recall@k**: the fraction of all relevant companies that appear in the top-k results, Recall@$k = \frac{|T_k(q) \cap R_q|}{|R_q|}$. This measures coverage, important for downstream re-ranking pipelines that operate on a retrieved candidate set. **Recall is the primary metric in this thesis**, following the priority set by Istari for this task. Whether the system finds the relevant companies matters more than the exact order they are ranked in.
- **NDCG@k** (Normalised Discounted Cumulative Gain): a ranking-aware metric that rewards systems for placing relevant companies higher in the list, using a logarithmic discount. It is reported as a secondary, supporting metric, reflecting the experience of a user who interacts primarily with the top results.

All metrics are averaged across all 101 queries. Precision and Recall are always reported at the same cut-off values so that the precision–recall trade-off at each depth is visible.

Reading Precision and Recall together. Every one of the 101 queries has exactly 1 000 relevant companies under the pseudo-relevance protocol, so at any cut-off $k \leq 1\,000$ the two metrics are exactly proportional to one another,

$$\text{Recall@}k = \text{Precision@}k \times \frac{k}{1\,000},$$

and only coincide once $k = 1\,000$, where the denominators become equal. A practical consequence is that Recall@k for small k can look misleadingly low in isolation. A system with a strong Precision@10 of 0.97 still shows a Recall@10 of only $0.97 \times 10/1\,000 \approx 0.0097$, not because the top-10 results are poor, but because 10 items can never cover more than 1% of a 1 000-company relevant set. Precision@k is therefore the more directly interpretable number at shallow cut-offs, while Recall@1 000 is the headline metric throughout this thesis, the one cut-off at which Recall stops being a rescaled copy of Precision and becomes a genuinely independent measure of total coverage.

Computational environment. All experiments are conducted on bwUniCluster 3.0, a shared high-performance computing (HPC) cluster accessed through the University of Mannheim.[1] Dense model encoding is performed on an NVIDIA A100 graphics processing unit (GPU) with 80 GB HBM2 memory, on a node equipped with two Intel Xeon Platinum 8358 CPUs (64 physical cores total). BM25 retrieval runs on the central processing unit (CPU) instead, since it does not involve any neural network computation.

[1] https://wiki.bwhpc.de/e/BwUniCluster3.0

Query latency measurements are performed on the same hardware and reported as the mean over 20 benchmark queries to obtain stable estimates. Latency is decomposed into three components (local query encoding, FAISS index search, and external network time) so that self-hosted and API-based methods are compared on equal terms.

Baseline Systems: Sparse and Dense Retrieval. **BM25**, introduced in Section 3.1, is included here as the sparse-retrieval baseline against which every *dense* (embedding-based, semantic) method in this chapter is compared.

The BM25 baseline is implemented with the `rank_bm25` Python library (Robertson and Zaragoza 2009), chosen for this thesis specifically because it provides the Okapi BM25 variant with well-established default hyperparameters ($k_1 = 1.5$, $b = 0.75$), avoiding the need for a separate hyperparameter search for what is only the lexical baseline. All 98 716 company summaries are tokenised using the punkt tokeniser from the Natural Language Toolkit (NLTK) (Bird et al. 2009) with lower-casing applied, and the resulting token lists are used to construct the BM25 inverted index. At query time, each query is tokenised identically and scored against the index. The top-1 000 companies by BM25 score are retrieved for each query and stored for evaluation. Index construction takes 28.7 seconds and requires no GPU.

Dense retrieval represents both the query and every candidate document as a fixed-length numerical vector (an *embedding*) produced by a neural network, such that texts with similar meaning end up as nearby vectors in that space, regardless of whether they share any exact words. This directly targets BM25's weakness. Two texts can be semantically related while sharing almost no vocabulary. Three dense retrieval baselines are evaluated first, covering a range of model sizes, computational requirements, and deployment constraints. All three follow the same two-stage pipeline introduced by Karpukhin et al. (2020). First, all company summaries are encoded offline into a dense vector index using FAISS (Johnson et al. 2019). Second, at query time, the query is encoded into a vector and the top-1 000 nearest neighbours are retrieved from the index. MiniLM, BGE, and OpenAI's model were chosen because they represent three deployment profiles a practitioner actually has to choose between. MiniLM is small and self-hosted, BGE is large and self-hosted, and OpenAI's model is a paid commercial API. Section 4.2.1 later broadens this comparison to eight further models to check whether a newer or larger model changes the conclusion.

MiniLM (all-MiniLM-L6-v2)

`all-MiniLM-L6-v2` is a lightweight sentence-transformer model (Reimers and Gurevych 2019) with 22 million parameters, producing 384-dimensional embeddings. It is distilled from a larger BERT model and fine-tuned on sentence similarity tasks. All 98 716 company summaries are encoded in a single batch pass, taking 0.6 minutes on the A100 GPU. A `faiss.IndexFlatL2` index (exact Euclidean search) is constructed over the resulting embeddings in 0.2 seconds. MiniLM does not require embedding normalisation.

MiniLM is included as the smallest and most computationally efficient dense baseline, providing a lower bound on dense retrieval quality and establishing whether a minimal

dense model is sufficient to outperform BM25.

BGE (bge-large-en-v1.5)

`BAAI/bge-large-en-v1.5` is a 335-million parameter BERT-large model developed by the Beijing Academy of Artificial Intelligence (Xiao et al. 2024), producing 1 024-dimensional embeddings. It is specifically optimised for dense retrieval through hard negative mining during fine-tuning, a training strategy in which the model is exposed to query–document pairs that are superficially similar but semantically distinct, forcing it to learn fine-grained semantic distinctions.

Encoding all 98 716 company summaries takes 7.0 minutes on the A100 GPU at a batch size of 256. BGE requires L2 normalisation of all embeddings (`normalize_embeddings=True`), as it is trained under the cosine similarity objective. Accordingly, a `faiss.IndexFlatIP` index (inner product, equivalent to cosine similarity for unit-normalised vectors) is used rather than `IndexFlatL2`.

At query time, each query is encoded independently and normalised before being passed to the FAISS index; the resulting query latency is reported alongside every other method's in Section 5.1.

OpenAI (text-embedding-3-small)

`text-embedding-3-small` is a commercial embedding model provided by OpenAI via their REST API. It produces 1 536-dimensional embeddings. Unlike the open-source models above, the architecture and training procedure are proprietary. Encoding all 98 716 company summaries is performed via batched API calls (batch size 500) and takes 9.7 minutes, of which the majority is network latency rather than model computation. The resulting embeddings are unit-normalised by the API, and a `faiss.IndexFlatIP` index is constructed locally. The total cost of encoding all 98 716 companies in this evaluation corpus is approximately $0.15 at the published pricing of $0.02 per million tokens.

This model is included as the commercial upper-bound baseline, testing whether a high-quality API-based embedding model outperforms the open-source alternatives evaluated above.

4.2 Phase II: Improving Retrieval

4.2.1 Extended Baseline Comparison

Section 4.1's three dense baselines already span a size and cost spectrum, from MiniLM's 22 million parameters to OpenAI's paid commercial API, and found the much smaller MiniLM ahead of both BGE and OpenAI, which were themselves effectively tied with one another. This raises a direct follow-up question. Does retrieval quality keep improving as model size and specialisation increase, and if so, is that improvement worth the added compute cost? Rather than sampling arbitrarily from the many embedding models available, the five chosen represent the two distinct strategies for building a retrieval

embedding model identified in Chapter 3 (Section 3.2): training a comparatively small encoder carefully and specifically for retrieval (BGE-M3, GTE-large), versus adapting an already-large, general-purpose language model into an embedding model through further contrastive fine-tuning (E5-Mistral-7B-Instruct, Linq-Embed-Mistral, SFR-Embedding-Mistral). These five were evaluated on the same 98 716-company corpus, all-fields text representation, and 101-query evaluation set described in Section 4.1. The first tier, mirroring the small-and-specialised strategy, stays close to BGE's scale (1 024-dimensional models, comparable parameter count) and comprises BGE-M3 and GTE-large. The second tier jumps to 7-billion-parameter models, an order of magnitude larger, but holds the base architecture and parameter count fixed across all three (E5-Mistral-7B-Instruct, Linq-Embed-Mistral, SFR-Embedding-Mistral all build on the same Mistral-7B backbone), varying only their fine-tuning data and objective, so any quality difference among these three specifically isolates the effect of fine-tuning from the effect of raw scale, rather than confounding the two:

- **BGE-M3** (`BAAI/bge-m3`): a multi-lingual, multi-functionality embedding model, 1 024-dimensional embeddings.
- **GTE-large** (`thenlper/gte-large`): a general-purpose text embedding model, 1 024 dimensions.
- **E5-Mistral-7B-Instruct** (`intfloat/e5-mistral-7b-instruct`): an instruction-tuned 7-billion-parameter Mistral-based embedding model, 4 096 dimensions.
- **Linq-Embed-Mistral** (`Linq-AI-Research/Linq-Embed-Mistral`): a 7B-parameter Mistral-based embedding model fine-tuned for retrieval, 4 096 dimensions.
- **SFR-Embedding-Mistral** (`Salesforce/SFR-Embedding-Mistral`): Salesforce's 7B-parameter Mistral-based retrieval embedding model, 4 096 dimensions.

4.2.2 HyDE and RRF

HyDE (Hypothetical Document Embeddings) is a query-expansion technique. Instead of embedding the user's short query directly, an LLM first generates several hypothetical, ideal-answer documents for that query, and their embeddings are averaged into a single, richer query vector. This thesis implements HyDE by asking a local LLM (`llama3.1:8b` via Ollama) to generate $N = 10$ stochastic (temperature $= 0.8$) hypothetical company descriptions that would ideally answer the query. The embeddings of these 10 descriptions are averaged into a single query vector, which is then searched against the MiniLM company index. A second stage reranks the top-50 HyDE results with a cross-encoder (`BAAI/bge-reranker-v2-m3`).

Reciprocal Rank Fusion (RRF) combines two or more separately produced rankings into one, by giving every document a score based only on its *rank position* in each input ranking (not its raw similarity score, which is not directly comparable across dif-

ferent retrieval methods), then summing those rank-based scores across methods:

$$\mathrm{RRF}(d) = \sum_{\text{method}} \frac{1}{k_{\mathrm{RRF}} + \mathrm{rank}(d)}$$

As an alternative to query expansion, BM25 and MiniLM rankings are fused directly using RRF with $k_{\mathrm{RRF}} = 60$ (Cormack et al. 2009). A second stage reranks the top-50 fused results with the same cross-encoder used for HyDE.

4.2.3 ANN Index Scaling to 2 Million Vectors

Approximate Nearest Neighbour (ANN) search trades a small amount of retrieval accuracy for a large speed-up, by searching an index structure that does not guarantee finding the exact closest vectors, unlike the exact brute-force search used everywhere else in this chapter so far. This section runs a first, synthetic-scale test of that trade-off. To test whether exact search (`IndexFlatL2`/`IndexFlatIP`) remains practical at production scale, the 98 716 real MiniLM company embeddings were padded with 1 901 284 synthetic filler vectors (real embeddings perturbed with Gaussian noise at 15% of the average vector norm), giving a 2-million-vector corpus. Exact search (`FlatL2`) was compared against two approximate indexes. IVF (`nlist`=4096, with `nprobe` $\in \{1, 8, 32, 128, 4096\}$) and HNSW ($M = 32$, with `efSearch` $\in \{16, 64, 256\}$) were both tested.

Recall@1,000 error analysis. To understand why exact MiniLM search does not achieve perfect Recall@1,000 (Section 5.1), every production-relevant company that MiniLM failed to retrieve within the top-1 000 was extracted and searched for at much greater depth ($k = 10\,000$). The resulting misses are classified and quantified in Section 5.2.3.

4.2.4 Corpus Quality: Platform Boilerplate

The duplicated "Bluehost.com" description found in Section 5.2.3 is one instance of a broader, previously unquantified data-quality issue affecting the `summary` field that every retrieval method in this thesis depends on. For a meaningful share of companies in the corpus, quantified directly in Section 5.2.4, the AI-generated summary describes the website hosting provider or website-building platform that the company's domain happens to run on, rather than the company's own business. This happens because many small businesses have a thin, template-driven web presence, so the process that generates each company's summary from its web content sometimes picks up generic platform language (for example, marketing copy for a web hosting provider or an e-commerce storefront builder) instead of any information specific to the company itself. A company using such a platform is then, in effect, indistinguishable in its summary text from every other company using the same platform, the mechanism behind the description duplication already observed in Section 5.2.3.

Detection approach. A classifier was built to flag boilerplate summaries automatically, using embedding similarity between each company's summary and a small, fixed reference set of around 300 known boilerplate templates, an approach chosen so that classifying any single company costs one embedding comparison against a fixed-size reference set, regardless of whether the underlying corpus is the 98 716-company evaluation set used in this thesis or the full multi-million-company GOI.

Oracle union recall ceiling. Before investing in a trained reranker or a fine-tuning pipeline, it is useful to know the best-case recall achievable by simply combining the candidate pools already produced by every method run so far, at zero additional encoding cost, since every channel's top-1 000 results per query are already cached on disk. This mirrors how pool coverage grows as more systems contribute to a pooled test collection (Zobel 1998), applied here to the retrieval channels themselves rather than to human relevance judging. This experiment computes, for every combination of the six available channels (MiniLM, BM25, BGE-all-fields, BGE-summary, OpenAI-large, Nomic), the recall of the *union* of their top-1 000 candidate sets against the top-1 000 ground truth.

4.2.5 Alternative Retrieval Architectures: SimLM and ColBERT

All dense baselines evaluated so far are general-purpose sentence-embedding models. **SimLM** (Wang et al. 2022) takes a different approach. It is a BERT-base (110 M parameter) encoder pre-trained specifically for the retrieval task itself, matching a short query to a longer passage via a representation-bottleneck objective, before any task-specific fine-tuning. It is included to test whether a model purpose-built for retrieval, but far smaller than the 7B-parameter models in Section 4.2.1, can match or exceed them.

SimLM implementation. The public checkpoint `intfloat/simlm-base-msmarco-finetuned` is used, following its documented convention rather than the all-fields concatenation used elsewhere in this thesis. The company name is encoded as a *title*, and the remaining fields (country, state, city, type, size, industry, summary, keywords) as the paired *body* text, using CLS-token pooling (not mean pooling) with L2 normalisation. Queries are encoded without any prefix, following the same model card.

Every method evaluated so far represents each company and each query as a single dense vector. **ColBERT** (Khattab and Zaharia 2020; Santhanam et al. 2022b) instead represents both as *sets of per-token* embeddings and scores a (query, company) pair using *MaxSim*. For every query token, the maximum similarity to any company token is taken, and these per-token maxima are summed. This *late-interaction* design is architecturally richer than single-vector dense retrieval, since no single fixed-size vector has to compress the entire meaning of a company description, and it is included to test whether that added expressiveness translates into higher recall on this task.

ColBERT implementation. The public checkpoint `colbert-ir/colbertv2.0` is used, encoding each company summary into a sequence of 128-dimensional per-token embed-

dings via the model's linear projection layer. Due to indexing and library constraints on the cluster used for this thesis, the official PLAID-indexed ColBERT pipeline was not used; instead, MaxSim scoring is implemented from scratch as a brute-force computation over all 98 716 companies for every query. This is computationally more expensive than an indexed implementation would be, but is exact rather than approximate, and is tractable at this corpus size.

4.2.6 Full-Pool Reranking and Learned Fusion Ranker

Section 4.2.2 showed that fusing BM25 and MiniLM with Reciprocal Rank Fusion, followed by reranking only the top-50 fused candidates, essentially reproduces MiniLM's own quality (NDCG@10 = 0.975) without improving Recall@1,000. Section 4.2.4 then showed that the *union* of several channels' top-1 000 candidate pools reaches a much higher oracle recall ceiling than any single channel. Together, these two findings motivate a different hybrid design. Rerank the *entire* union pool of several dense channels with a cross-encoder, rather than fusing rankings and reranking only a shallow, fixed-size slice of the result.

Full-pool reranking implementation. For each query, the top-1 000 results from a set of dense channels are combined by simple set union (no rank fusion, no truncation), giving an unranked candidate pool that is typically 1 300–1 800 companies. A cross-encoder reranker, `BAAI/bge-reranker-v2-m3` (BAAI 2024), then scores every (query, candidate) pair in the full pool directly, and the top-1 000 by reranker score are kept as the final result. Four variants are compared, adding one channel at a time:

- **2-way**: MiniLM + Linq-Embed-Mistral.
- **3-way**: 2-way + GTE-large.
- **4-way**: 3-way + OpenAI-large.

Reranker ablation: isolating the bottleneck. Given the channel-saturation finding in Section 4.2.6, the natural next question is whether the specific reranker model used throughout this chapter, `BAAI/bge-reranker-v2-m3`, is itself limiting recall, or whether reranker choice matters little once the candidate pool is fixed.

Reranker ablation implementation. `BAAI/bge-reranker-v2-m3` (built on an XLM-RoBERTa backbone) is replaced with `mixedbread-ai/mxbai-rerank-large-v1` (Mixedbread AI 2024) (built on a DeBERTa-v2 backbone, an architecturally distinct, comparably-sized cross-encoder), applied to the *exact same* 3-way union pool from Section 4.2.6. Only the reranker model changes; the candidate pool, corpus, and queries are identical.

Every hybrid pipeline so far combines *frozen*, independently pretrained components. Candidate lists are pooled by simple union, and a pretrained cross-encoder, never fit to this task's own queries or labels, reranks the result. This section instead treats fusion as a supervised **learning-to-rank** problem. A model is trained directly on this thesis's own

101 queries and their pseudo-relevance labels to learn how to combine multiple retrieval signals, rather than relying on a fixed, hand-designed combination rule.

Formal setup. For each (query, candidate) pair, the model receives a ten-dimensional input feature vector $x \in \mathbb{R}^{10}$ (the similarity scores, inverse ranks, reranker score, and channel count described below) and predicts a single output $\hat{y} \in [0, 1]$, the estimated probability that the candidate is relevant to the query. The training target $y \in \{0, 1\}$ is the pseudo-relevance label from Section 4.1 (1 if the candidate is in production's top-1 000 for that query, 0 otherwise). Both model variants are trained to minimise binary cross-entropy (log loss), $\mathcal{L}(y, \hat{y}) = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$, averaged over all training (query, candidate) pairs, the standard objective for a probabilistic binary classifier and the default loss for both `scikit-learn` estimators used here.

Learned fusion ranker implementation. For every (query, candidate) pair in the same 3-way union pool used in Section 4.2.6 (with BM25 added as a fourth, lexical, signal), ten features are computed: the raw similarity score and inverse rank (1/rank) from each of MiniLM, Linq-Embed-Mistral, GTE-large, and BM25; the `BAAI/bge-reranker-v2-m3` score from Section 4.2.6 (included as one input feature among ten, not as the final decision rule); and a count of how many channels retrieved the candidate at all. Two models are trained on these features using `scikit-learn` (Pedregosa et al. 2011), a logistic regression and a histogram-based gradient-boosted decision tree ensemble (GBDT). Crucially, both are trained and evaluated using 5-fold cross-validation grouped by `query_id`. For each fold, the model is fit on candidates from 80% of queries and scores held-out candidates from the remaining 20%, so that no query is ever scored by a model that has seen its own label during training. Final metrics are computed by concatenating the held-out predictions from all five folds, giving a complete, leakage-free evaluation over all 101 queries.

4.3 Phase III: Evaluation Validity

The pseudo-relevance protocol used throughout this thesis (Section 4.1) treats a company as relevant if and only if it appears in production's own results for a query. This is a reasonable practice when no human-annotated relevance judgements exist (Section 3.7), but it carries a specific risk for the learned fusion ranker (Section 4.2.6): a model trained directly on labels derived from production's own output may partly be learning to reproduce whatever idiosyncrasies that particular system has, rather than genuine relevance. This section reports a small-scale attempt to check whether the learned fusion ranker's key interpretive finding, that MiniLM's inverse rank is the most informative feature, survives replacing that circular ground truth with an independently constructed one.

4.3.1 Building the Silver Standard

This section builds this thesis's *silver standard*[2], a pooled, multi-judge, human-adjudicated relevance standard used throughout the remainder of this thesis.

Pooling. Following the pooling methodology established for the Text REtrieval Conference (Voorhees 2002; Zobel 1998), the top-20 candidates per query were taken from six sources. These were the frozen production ground truth, a freshly fetched sample from Istari's live v2 search API, BM25, GTE-large, ColBERT, and the learned fusion ranker's own output, then unioned and deduplicated by domain. Applied across all 101 queries, this yields 8 309 pooled candidates, an average of 82.3 unique candidates per query, a substantially smaller and more tractable set than the original 1 000-per-query pool.

Trust filtering. Before any relevance judging, every pooled candidate's summary was checked against the boilerplate detector from Section 4.2.4, reusing its already-cached reference templates. This flagged 487 of 8 309 candidates (5.9%) as untrustworthy, which were excluded from judging entirely, so that a summary describing the wrong entity could not poison a relevance label built on top of it.

Multi-judge relevance labelling. Each remaining, trustworthy candidate was independently judged by two large language models (OpenAI's `gpt-4o-mini` and Anthropic's Claude Haiku), on a three-level scale (0 = not relevant, 1 = partially relevant, 2 = highly relevant), following the LLM-as-judge approach validated at scale by Thomas et al. (2024). This cheaper judge pair was chosen deliberately for this stage. The initial pilot and active-learning expansion (Section 4.3.2) required judging over a thousand candidates, and a lower per-candidate cost made judging at that volume affordable at all. This choice is validated rather than assumed. The blind calibration check below and a direct kappa comparison against a stronger judge pair (Section 5.3.1) confirm the cheaper pair reaches "substantial" rather than "almost perfect" agreement, a real but bounded gap that motivated upgrading to stronger judges (`claude-sonnet-5`, `gpt-5.4`, and Google's Gemini) for the later spot-checks and the production-independent evaluation (Sections 5.3.2, 4.3.3). Where the two judges agreed, that label was accepted directly. Where they disagreed, the case was resolved by manual review, performed by the author. The resulting standard is therefore independent of the production system, but not fully independent of the researcher, a limitation discussed in full in Section 5.3.5.

Blind calibration of the judges. Resolving disagreements by reading both judges' stated reasoning risks anchoring the human reviewer towards whichever judge's framing is read first. To check this, a separate blind sample of 20 candidates that neither judge

[2] This thesis calls the pooled, multi-judge, human-tie-broken labels built in this section a *silver standard* rather than a *gold standard*, since LLM judges with only minimal human checking do not meet the bar traditional IR evaluation reserves for that term. The larger, classifier-distilled set built from it in Section 4.3.2 is correspondingly renamed from *silver* to *bronze*, one trust tier below.

had yet seen was labelled by the author with no access to either judge's output, then scored independently by both judges. OpenAI matched the blind label 75% of the time and Claude 80%, close enough that neither judge should be treated as categorically more reliable than the other on this task.

Scope of this pilot. Labelling all 101 queries with two paid LLM judges directly was not attempted, given the per-candidate cost of LLM judging (quantified in full in Section 4.3.2). A 5-query pilot was completed first instead, to validate the labelling approach at a small, affordable scale before committing to a larger expansion. Of 430 candidates judged, 360 (83.7%) received unanimous agreement between the two judges. Of the remaining 70 disagreements, 59 were resolved by manual review and 11 were excluded outright for insufficient evidence (for example, unreachable domains or placeholder websites with no verifiable content), rather than being forced into a label the available information could not support. This yields **419 silver-labelled candidates** across 5 queries, distributed as 8 not relevant, 119 partially relevant, and 292 highly relevant.

4.3.2 Retraining and Expanding the Silver Standard

The exact architecture from Section 4.2.6 (the same ten features, the same 5-fold cross-validation grouped by `query_id`, here equivalent to leave-one-query-out since exactly five queries are covered) was retrained using the 417 of 419 silver-labelled candidates that matched an existing feature row from the original candidate pool. The binary classification target used the strict threshold `gold_label` $= 2$ (highly relevant only, 291 positive vs. 126 negative examples); the more permissive threshold `gold_label` ≥ 1 leaves only 8 negative examples across the whole pilot, too few for cross-validated coefficients to be estimated reliably. Ranking quality is reported as NDCG@5/10/20 rather than the deeper cutoffs used elsewhere in this thesis, since this pilot's candidate pool is only around 80 silver-labelled candidates per query, an order of magnitude smaller than the top-1,000 pools evaluated in earlier sections, so a cutoff of 1,000 would simply score the entire pool rather than isolate top-of-list quality.

No gold-standard relevance data exists for this task at any scale to build on. Istari's full 20-million-company database has never been exhaustively human-labelled by anyone, so this thesis treats labelling as an incremental process rather than a one-time effort. A small, independently judged set is verified first (Section 4.3.1), and how much further it is affordable and worthwhile to extend then becomes a concrete decision rather than an assumption that the answer is either "all of it" or "none of it". Extending independent LLM judging to every candidate the retrieval systems in this thesis actually produce is not affordable. This estimate is not a guess. It scales the per-candidate API cost actually incurred while judging the 419-candidate pilot (Section 4.3.1) up to larger volumes at different pooling depths. The original fusion ranker's own candidate pool (Section 4.2.6) covers 173 262 (query, candidate) pairs across all 101 queries. Judging every one of those at the pilot's observed per-candidate cost would cost approximately $22 000, rising towards $28 000 at a deeper pooling depth approaching exhaustive judging of every corpus company against every query, and the majority of those pairs are companies no

retrieval channel ever considered plausible, judging them would mean paying to confirm the obvious.

Instead, the 419 silver-labelled candidates from Section 4.3.1 are used to train a final classifier (the same architecture and feature set as Section 4.2.6, fit on all 419 labels rather than held out via cross-validation, since this model is intended for deployment rather than evaluation). This classifier is then applied to all 173 262 existing candidate rows, which already have the required ten features computed, at no additional retrieval or labelling cost. This produces two explicitly distinct label tiers, following standard practice in weak and distant supervision. These are a small **silver** tier (419 candidates, LLM-judged and human-adjudicated) and a much larger **bronze** tier (the remaining candidates, model-predicted rather than independently judged). The two tiers are not merged or treated as equally reliable anywhere in this pipeline.

Validating the bronze tier. To estimate how trustworthy the bronze labels actually are, a random sample of 150 bronze-labelled candidates is independently judged by two more capable models, `claude-sonnet-5` and `gpt-5.4` (an upgrade from the `claude-haiku-4-5`/`gpt-4o-mini` pair used for the original silver standard, chosen after confirming via the blind calibration check in Section 4.3.1 that the cheaper pair was not itself the limiting factor). Only candidates where both stronger judges unanimously agree are counted, disagreements are excluded from the spot-check rather than guessed at, the same principle used throughout Section 4.3.1. This validation costs under $1, compared to the $22 000+ a full re-judging pass would require.

Rather than judging a random or exhaustive sample of the remaining 96 uncovered queries, the silver standard is expanded using *uncertainty sampling*. The distillation classifier's own predicted probability is used to find, for every one of the 101 queries, the ten candidates it is least confident about (predicted probability closest to 0.5), and only those are sent for judging. This targets exactly the candidates most likely to teach the classifier something it does not already know, rather than spending judging budget on cases it already calls correctly. The judging prompt is enriched at the same time with company metadata that was previously collected but never shown to the judges. This includes `organization_type`, `organization_size`, `state`, `district`, `municipality`, and `nace_code`, alongside the original name, country, and summary fields, giving the LLM judges more grounding for cases a summary alone leaves ambiguous. This active-learning approach is well established in the machine learning literature (Settles 2009), and combining LLM-based and human-provided relevance judgements to build a test collection under a limited labelling budget mirrors the approach of Takehi et al. (2024), who propose a similar strategy for balancing manual and LLM annotation effort in IR test collection construction. Uncertainty sampling specifically for building IR test collections, rather than for training a downstream classifier, has direct precedent in Rahman et al. (2020), who show that selecting documents for human judgement by a system's own uncertainty reduces the labelling budget needed to reach a reliable test collection, the same logic applied here to selecting candidates for LLM judgement instead. This is not a universally settled question, however. Althammer et al. (2023)

find that, for the specific task of selecting training data to fine-tune a neural ranker, active learning strategies did not outperform random selection once labelling cost was accounted for. This thesis's use case differs in an important respect that plausibly explains why uncertainty sampling still pays off here. The goal in Section 4.3.2 is not to train the best possible ranker directly, but to close a measured, diagnosed coverage gap in the underlying silver standard (Section 5.3.2), where deliberately targeting the classifier's most uncertain, least-covered queries is exactly the failure mode identified. Xia et al. (2025)'s recent survey of LLM-based active learning situates this thesis's approach within a fast-moving area of the literature, where using an LLM as the labeller within an uncertainty-sampling loop, rather than only as the model being trained, is an increasingly common pattern that this thesis's own active-learning pass follows.

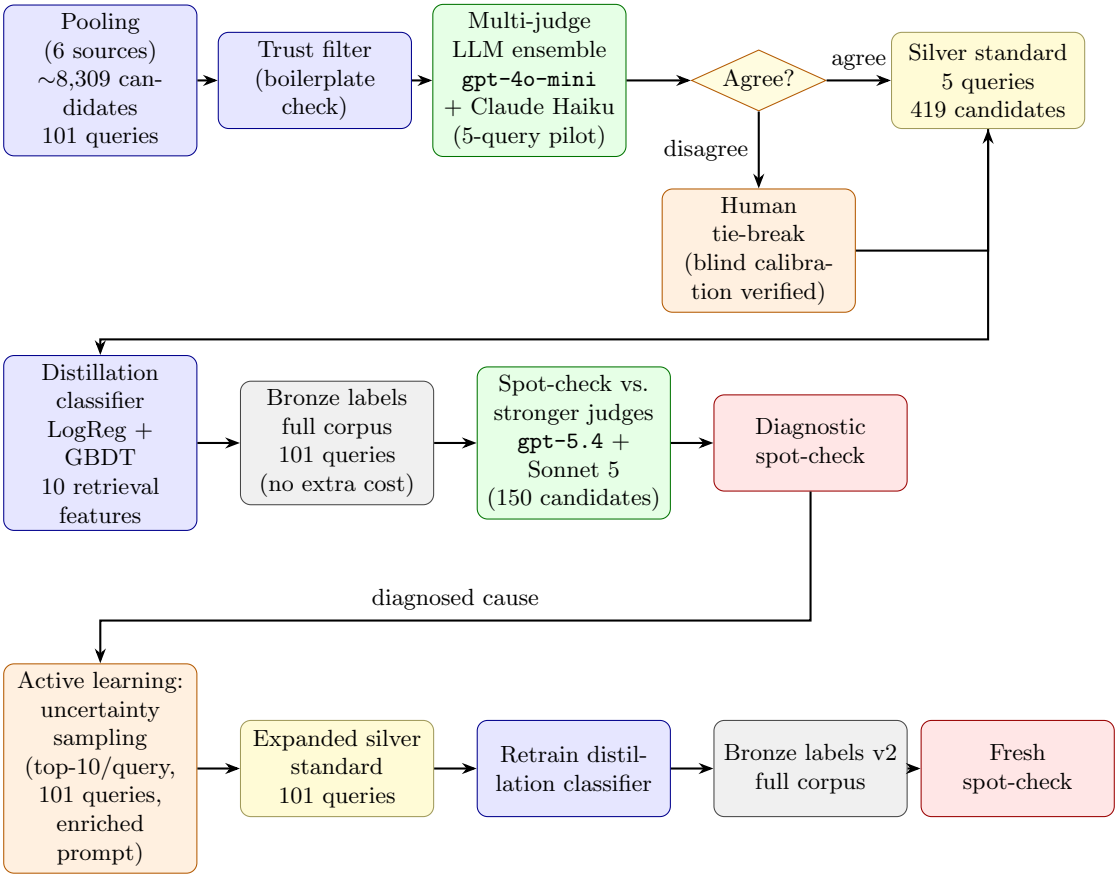


Figure 4.2: The silver and bronze standard construction pipeline, from pooling through active-learning expansion.

Figure 4.2 summarises the full pipeline described across this section.

The shallow, roughly uniform depth left by Section 4.3.2's expansion (10-14 silver-labelled candidates per query) fixes out-of-distribution generalisation but is too thin to report reliable per-query Recall@k/NDCG@k as an evaluation benchmark, deeper

judged pools reduce the variance of such metrics (Section 4.5.1 discusses this further). Rather than deepening all 101 queries, this thesis selects a set of 14 queries for methodological diversity beyond simple topic variety, covering geographic coverage not tested by the original pilot, the BM25 vocabulary-mismatch failure mode ("venture capital firms"), adjacent-category disambiguation (e.g. pharmaceutical/biotech, investment/wealth management), acronym-heavy queries, and niche or emerging industries.

Unlike the one-shot batch in Section 4.3.2, this used genuinely iterative active learning. Each round labelled a batch, retrained the classifier on everything labelled so far, re-ranked the *remaining* candidates with the *updated* classifier, and repeated, since uncertainty estimates go stale as soon as new labels are added. A small fixed validation set (100 candidates, judged once with the stronger judge pair and reused for free across every round) tracked whether each additional round was still worth its cost.

Refreshing the bronze tier at no additional cost. The combined 2,269-candidate silver standard (1,429 from Section 5.3.2 plus 840 from this section) was used to retrain the distillation classifier from Section 4.3.2 once more (2,267 of the 2,269 matched an existing feature row, the same small gap noted in Section 4.3.2), and applied to the existing candidate pool to produce up to 1,000 bronze-labelled candidates per query, capped rather than kept at the full pool size, and with every silver-verified candidate guaranteed a place regardless of its predicted probability rank. This required no new retrieval, encoding, or judging. Every one of the 101 queries already has at least 1,022 candidates with retrieval features computed (median 1,661), so the only new work is classifier inference. All 101 queries now have exactly 1,000 candidates each (101,000 rows total, 2,267 of them silver-verified), the deepest per-query coverage produced so far in this thesis, though still bronze-tier and bounded by the classifier's demonstrated accuracy (Section 5.3.2) rather than independently verified.

Quantifying how far production's own ranking diverges from the independent silver/bronze set. Production's own top-1,000-per-query ranking, has exactly the same shape as the refreshed bronze set above (101,000 rows, 1,000 per query, identical `query_id/domain` keys), making a direct, row-level comparison possible with no new judging or encoding cost. Four checks quantify what has, until this point, mostly been argued qualitatively in this thesis. They measure how much production's own output actually diverges from an independently verified standard.

A direct ranking comparison: production vs. an independently cross-validated ranker. Section 4.3.2 shows production's ranking *differs* from the independent silver/bronze set; this section asks whether it is actually *worse* at finding truly relevant companies, a stronger and more specific claim. Production's own top-1,000-per-query ranking is compared against a ranking produced by the fusion-ranker classifier trained with leave-one-query-out cross-validation (the same principle as Section 4.3.2, extended from 5 to all 101 queries: for every query, the classifier is trained on every *other* query's silver labels only, so no ranking is ever produced by a model that saw that query's own labels).

All 101 folds train in under 15 seconds. Both rankings are scored against the same silver-verified relevant set (`gold_label` $= 2$) using Recall@k/Precision@k/NDCG@k at $k \in \{10, 50, 100, 300, 500, 1000\}$.

The explanation offered above, that production's advantage in Section 4.3.2 is a candidate-pool artefact rather than a genuine quality gap, was argued qualitatively. This section makes it concrete and measurable instead, following a specific critique of the evaluation methodology. Every precision, recall, and NDCG figure reported in this thesis implicitly treats an unjudged candidate as irrelevant, which punishes a system exactly when it retrieves something good the pooling process happened to miss. If different systems' top-k results carry very different fractions of already-judged candidates, their scores are not measuring the same thing, and are not directly comparable without knowing that fraction.

Method. For six systems, production's own ranking; the leave-one-query-out cross-validated fusion ranker from Section 4.3.2; and each of the four individual retrieval channels' own top-1,000 ranking within the same candidate pool (BM25, MiniLM, GTE-large, Linq-Embed-Mistral), label coverage is computed as the fraction of a query's top-k results that carry any silver label at all, relevant or not, independently of whether those labels indicate relevance. `bpref` (Buckley and Voorhees 2004), a metric introduced at TREC specifically for evaluation under incomplete judgements, is computed alongside the usual Recall@1,000. Rather than treating an unjudged document as confirmed non-relevant, bpref only counts judged non-relevant documents when penalising a relevant document's rank position, so a system is never docked for retrieving something genuinely unjudged rather than genuinely irrelevant.

4.3.3 A Production-Independent Test

Section 4.3.2's comparison could not yet support the claim that this thesis's ranking outperforms production, because the silver standard it was scored against was partly pooled from production's own output. This section closes that gap directly, using the random, production-independent 300,000-company sample introduced in Section 4.5.1 (`source == ''new''` in the combined pool, as opposed to `''existing''` for the original, potentially production-entangled corpus).

For the same 19 deep-coverage queries used throughout this thesis's rigorous comparisons (a leakage-free held-out set never used for hyperparameter or model selection, Section 4.4.1), candidates are pooled from four independent retrieval channels (BM25, MiniLM, GTE-large, Linq-Embed-Mistral), searched exclusively over `source == ''new''` companies so the pool is never diluted by the already-curated existing corpus, then trust-filtered the same way as the original silver standard (Section 4.3.1). Every candidate in this pool is, by construction, a company production was never asked to rank for any of these queries, so production's own output cannot have influenced which candidates end up here, this removes the candidate-pool bias identified in Section 5.3.3 by design, not by argument. Trustworthy candidates are then judged by two strong LLM judges from two different labs, Claude Sonnet 5 and Google's Gemini 3.6

Flash, chosen instead of the originally planned GPT-5.4 pairing specifically because this thesis's own OpenAI baseline (`text-embedding-3-small`, Section 4.1) is itself under evaluation, so an OpenAI judge risks the same-family bias this thesis otherwise avoids by construction. Disagreements are resolved by hand, the same tie-breaking principle used for every other silver-standard pass in this thesis.

Because production has no ranking data at all for `source == ''new''` companies (only ever covers the original corpus), a symmetric production-vs-ours ranking comparison is not even defined here. This section runs a different, meaningful test instead. Do the resulting silver-verified relevant companies, independently and blindly confirmed, get found and ranked well by this thesis's own retrieval pipeline, searching the full, realistic corpus across all four scaling tiers (Section 4.5.1)? This directly tests the coverage-gain claim from Section 4.3.2 on a candidate pool that is clean of circularity by construction.

The resulting judgements, and how they were used to build the confirmed-relevant set this test is scored against, are reported in Section 5.3.4.

Checking whether the classifier's confidence is honest. The accuracy figures reported throughout Section 5.3.2 measure whether the classifier's predicted label (relevant/not, at the 0.5 cutoff) is correct. A separate question is whether its predicted *probability* is honest, when it reports 90% confidence, is it right about 90% of the time. This matters because the overwhelming majority of the bronze tier was never individually spot-checked; trusting the classifier's confidence at face value depends on it being well calibrated, not just accurate at one threshold. Using the same leave-one-query-out cross-validated predictions as Section 4.3.2, predicted probabilities for all 2,267 silver-labelled candidates were binned into deciles and compared against the actual relevant rate within each bin.

4.3.4 Silver-Standard Coverage Across the Ranking

Before sending ranked candidate lists to an external reviewer at ISTARI.AI for a relevance-label review, it is worth checking where the silver labels already collected sit within each query's own ranked output, since that determines whether the top of the ranking can already be called trustworthy or whether it is still, in effect, unverified.

Two representative queries. Query 1 ("software companies") and query 92 ("wealth management firms") were chosen to send out, the former for its comparatively large existing label count, the latter for having the largest coverage gap identified in Section 5.3.3.

For each query, the classifier-ranked top-1,000 candidates from Section 4.3.2's bronze-label pipeline were split into five rank bands (1–100, 101–300, 301–500, 501–700, 701–1,000), and every candidate already carrying a verified silver label was located within its band.

Implication for the external review. A curated 100-candidate export per query was prepared for the external review, drawing 20 candidates from each of the same five

rank bands, prioritising candidates absent from production's own top-1,000 for that query, to test directly whether the pipeline surfaces genuinely relevant companies production misses. Already-labelled candidates remain at their natural position, unflagged, so agreement with this thesis's own silver labels on those rows was intended as a free, blind inter-rater consistency check. The reviewer was given written guidelines matching the standard already used for LLM judging (Section 4.3.1), also folded directly into the judge prompt from that point forward so that human and model judgements apply the same criteria.

4.4 Phase IV: Learned Retrieval Revisited

4.4.1 Hyperparameter Tuning and Neural Network Fusion Ranker

Every fusion ranker result reported so far, the original model (Section 4.2.6), the pooled-silver-standard retraining (Section 4.3.2), and the cross-validated ranking comparison against production (Section 4.3.2), uses the same hand-picked `HistGradientBoostingClassifier` configuration (`max_iter=150`, `max_depth=4`, `class_weight="balanced"`), chosen once early on and never systematically tuned against the silver standard. This section addresses that gap with a randomised hyperparameter search.

The search space covers `max_depth` (2 to 8, or unlimited), `max_iter` (50 to 300), `learning_rate` (0.03 to 0.3), `min_samples_leaf` (5 to 30), and `l2_regularization` (0.0 to 2.0); `class_weight="balanced"` is kept fixed throughout, since it is an established, justified design choice rather than a model-capacity hyperparameter. 40 combinations are sampled at random (fixed seed for reproducibility).

Avoiding leakage in model selection. Selecting the best combination using the same cross-validated metric later reported as the final result would bias that result upward (the "winner's curse" in model selection). A combination is not a fair, unbiased estimate of unseen-data performance if it was chosen *because* it scored best on exactly the data used to report it. To avoid this, the search itself is restricted to the 82 queries without deep silver coverage, evaluated with `LeaveOneGroupOut` cross-validation grouped by `query_id` within that 82-query pool only, scored by mean Recall@1,000 and NDCG@1,000. The 19 deep-coverage queries (5-query pilot plus 14 headline queries) are never touched during the search. Once the best combination is fixed, it is evaluated fresh with a full `LeaveOneGroupOut` pass over all 101 queries, the same protocol used for the untuned baseline (Section 4.3.2). The 19-deep-query subset of that final evaluation is a genuinely leakage-free comparison against the untuned baseline, since those queries never influenced which hyperparameters were chosen; the all-101-query subset is reported alongside it for comparability with earlier sections, but should be read as directional rather than strictly unbiased, since the other 82 queries did inform the search.

Every fusion-ranker result so far, tuned or not, uses `HistGradientBoostingClassifier`, a tree ensemble. This section tries a genuinely different model class on the same ten features, a small multi-layer perceptron (MLP), with its own hyperparameter search

over architecture (hidden layer sizes and depth), optimisation (learning rate), and regularisation (dropout, weight decay), asking whether a neural network does any better than the tuned tree ensemble from Section 4.4.1.

As with the GBDT and logistic-regression models (Section 4.2.6), the MLP takes the same ten-dimensional feature vector $x \in \mathbb{R}^{10}$ per (query, candidate) pair as input and is trained against the same pseudo-relevance target $y \in \{0, 1\}$. Each hidden layer applies a linear transformation followed by a ReLU activation, $\mathrm{ReLU}(z) = \max(0, z)$, and dropout at the searched rate; the final layer is a single linear unit producing a raw logit z, with no output activation applied directly. The network is trained to minimise a class-weighted binary cross-entropy loss applied to that logit, $\mathcal{L}(y, z) = -\left[w_y \log \sigma(z) \cdot y + \log(1 - \sigma(z)) \cdot (1 - y)\right]$ where σ is the sigmoid function and w_y up-weights the minority (relevant) class inversely to its frequency in the training fold, implemented as `BCEWithLogitsLoss` with a `pos_weight` argument in PyTorch, optimised with AdamW.

Unlike the GBDT search, full `LeaveOneGroupOut` cross-validation per hyperparameter combination is too expensive here, each fold trains a network from scratch rather than fitting a near-instant tree, so the search phase uses a single, fixed 80/20 split of the 82 tuning-only queries (66 train, 16 validation), never touching the 19 held-out deep queries; this is a standard, well-justified simplification for neural hyperparameter search. 20 combinations are sampled at random over hidden layer sizes ($\{16\}$ to $\{128, 64, 32\}$), dropout (0 to 0.5), learning rate (10^{-4} to 5×10^{-3}), and weight decay (0 to 10^{-2}), each scored by NDCG@1,000 on the 16 validation queries. Two details are specific to neural networks that a tree ensemble does not need. Features are standardised (zero mean, unit variance, scaler fit on the training split only) before being fed to the network, since gradient descent, unlike a tree split, is sensitive to feature scale, and the loss function is a class-weighted binary cross-entropy (weight inversely proportional to class frequency), mirroring the `class_weight="balanced"` convention used by every other classifier in this thesis. Once the best combination is fixed, it is evaluated with the same full `LeaveOneGroupOut` protocol over all 101 queries used everywhere else, so the comparison against the tuned GBDT on the 19 leakage-free deep queries is direct.

4.4.2 Fine-Tuning the Reranker

Every use of `BAAI/bge-reranker-v2-m3` throughout this thesis, in the hybrid reranking pipeline (Section 4.2.6) and as one of the ten fusion-ranker features, treats it as a frozen, pretrained component, never fit to this thesis's own queries or labels. This section fine-tunes it directly on the silver-labelled (query, candidate) pairs and checks whether that improves ranking quality, reusing the same 82-tuning/19-held-out query split as Section 4.4.1 rather than introducing a new one, so the two experiments stay directly comparable and the held-out queries remain untouched by either.

The reranker is fine-tuned as a regression problem. For each silver-labelled (query, candidate-summary) pair from the 82 tuning-only queries (∼1,000 examples), the target score is `gold_label`/2 (mapping 0/1/2 to 0.0/0.5/1.0, matching the reranker's own continuous score output), trained with mean squared error (MSE) loss, AdamW, learning

rate 2×10^{-5}, batch size 32, for 4 epochs. This is a small dataset for a ∼568M-parameter model, a real overfitting risk flagged explicitly rather than glossed over. Both the original pretrained reranker and the fine-tuned version are then evaluated on the same task the reranker actually performs in this pipeline. This means reranking each held-out query's full candidate pool from the union-pool feature set (Section 4.2.6, ∼1,000-2,900 candidates per query) and scoring the result by Recall@k/NDCG@k against the silver-verified relevant set, restricted to the 19 deep-coverage queries so the comparison is leakage-free by the same logic as Section 4.4.1.

A second attempt: hard negatives and a pairwise ranking loss. Rather than abandoning fine-tuning, the diagnosed cause was addressed directly. Each training query's negative pool was supplemented with candidates sampled from its own retrieval pool that were never silver-labelled at all (a standard, well-justified assumption given how sparse true relevance is in a raw candidate pool), giving 1,782 (positive, negative) training pairs across 79 tuning queries, versus only 19 explicit negatives out of 820 individual examples before. Training switched from MSE regression on an absolute score to a pairwise margin ranking loss, directly optimising for whether a relevant candidate outscores a non-relevant one for the same query, the thing Recall@k/NDCG@k actually measure, rather than an absolute score value.

4.5 Phase V: Scalability

4.5.1 Scaling to a Larger Corpus

The 98 716-company corpus used throughout this thesis is not just query-gatekept (Chapter 1), it is also small relative to Istari's real production dataset. The thesis title's "at scale" claim needs a genuine scalability test, does the retrieval pipeline's quality and latency hold up as the pool of companies to search grows substantially larger, not just a claim resting on the existing corpus size.

A fully random sample. A collaborator at Istari provided a fully random, query-independent sample of 300 000 companies drawn from the underlying database, deliberately not filtered or weighted towards any of the 101 queries used elsewhere in this thesis. Randomness here matters methodologically. Using this sample to check whether a retrieval system's index or corpus omits genuinely relevant material, rather than to build more query-specific training data, follows the same logic as random-sampling approaches used to estimate search engine index coverage (Bar-Yossef and Gurevich 2008), applied here to check whether the retrieval pipeline still finds known-relevant companies once far more distractor companies are added, rather than to estimate absolute index coverage as in that work.

Data quality. The sample was profiled and lightly cleaned. Notably, only 0.56% (1 691 companies) overlapped with the existing 98 716-company corpus, confirming the sample is genuinely independent rather than a re-draw of existing data. No duplicate

domains, no empty summaries, and no literal-string placeholder values were found. State/district/municipality fields were missing 15.7%/60.4%/75.4% of the time respectively, expected sparsity given inconsistent administrative-division reporting across countries, not a data quality defect. A small fraction of rows (1.03%) shared an exact-duplicate summary with another row, flagged rather than dropped, since a shared summary does not necessarily indicate an invalid company record.

Nested corpus tiers. Rather than four independently drawn samples of increasing size, the scaling corpus is built as four *nested* tiers. These are 100 000, 200 000, 300 000, and ∼397 000 (all available) companies, each strictly containing the previous tier plus additional randomly drawn companies from one fixed shuffle. The existing 98 716-company corpus, which holds all silver-labelled data used for evaluation, is included in every tier without exception, so that a change in a retrieval metric between tiers can be attributed to the added distractor companies, not to silver-labelled companies being dropped from the search pool. This nested design keeps the four tiers directly comparable to one another, following the same logic that motivates careful control of pooling depth in retrieval evaluation more broadly (Lu et al. 2016).

Encoding. Every company in the combined pool (∼397 025 companies) is encoded exactly once with each embedding model, then subset per tier, avoiding redundant computation. Two exceptions apply. BM25's index is rebuilt separately for each tier, since its term weights depend on the whole indexed collection rather than a fixed per-company encoding. The pretrained reranker (`bge-reranker-v2-m3`) is not part of this step at all, it scores candidates at retrieval time only, after first-stage retrieval, regardless of corpus size.

Scope: beyond the three fusion-ranker features. The scaled encoding was not limited to the three models already inside the trained fusion ranker (MiniLM, GTE-large, Linq-Embed-Mistral). Every embedding model evaluated as a baseline earlier in this thesis (BGE-large, BGE-M3, Nomic, E5-Mistral, SFR-Mistral, SimLM) and ColBERTv2 was also encoded at the new scale.

The motivation is the same circularity concern raised in Section 4.3.2. The original baseline comparison was evaluated entirely against the circular `goi_search_results.json` ground truth, before the independent, pooled silver standard existed. It has never been rechecked against that standard. The fusion ranker's own feature-importance finding was rechecked this way, and it partially reversed (Section 5.3.2).

Re-scoring every baseline model's already-computed rankings against this standard is left as a planned follow-up (Chapter 7).

ColBERT is architecturally distinct from every other model here. It stores one embedding per token rather than one vector per company ('doc_embs.pt', shape $[N, 192, 128]$, roughly 19.5 GB at this scale versus 4.85 GB for the original 98,716-company run), so it cannot simply be subset into corpus-size tiers the same way a single dense vector

per company can. It is included nonetheless, for completeness against every method previously evaluated.

The combined pool and nested tier membership have been constructed, and encoding is complete. All 10 models (MiniLM, GTE-large, BGE-large, BGE-M3, Nomic, Linq-Embed-Mistral, E5-Mistral, SFR-Mistral, SimLM, ColBERTv2) have produced a full embedding file covering all ∼397 025 companies.

The scaling evaluation asks whether Recall@k/NDCG@k and query latency hold up as the corpus grows from 100K to ∼400K companies. It is restricted to the 19 queries with the deepest, most reliable silver-standard coverage (the 5-query pilot plus the 14 headline queries, Section 4.3.2), so the resulting metrics are not read from an unreliably shallow label set. BM25 and the three embedding models already inside the trained fusion ranker (MiniLM, GTE-large, Linq-Embed-Mistral) are evaluated at every tier using exact brute-force search. ANN is treated separately in Section 4.5.2. Every silver-relevant candidate for the 19 deep queries is present in all four tiers by construction, confirmed directly (906 of 906 relevant candidates found in even the smallest, 100K, tier), so any change in these metrics across tiers is attributable to the growing pool of distractor companies rather than relevant companies dropping out of the search pool.

4.5.2 Approximate Nearest Neighbour (ANN) Index Tuning

The thesis proposal commits explicitly to approximate nearest neighbour search ("supports scalable top-k retrieval using approximate nearest neighbor search"; "trade-off between retrieval accuracy and latency under different ANN index configurations"). Section 4.2.3 took a first pass at this question early in the thesis, using a synthetic 2-million-vector corpus (real embeddings padded with Gaussian-noise-perturbed filler vectors) and found a 6.5–6.6× speedup with no meaningful quality loss. Every other result in this thesis, including the scaling evaluation above, still uses exact brute-force search on real data, a full dot product against every company in the tier. This section repeats the ANN question on the real, nested 100K–∼400K corpus built for the scaling evaluation, rather than a synthetic one, to check whether the earlier finding holds at genuine (if still sub-20-million) scale.

A FAISS (Johnson et al. 2019) HNSW index is built over GTE-large's embeddings (the strongest all-around performer from the scaling evaluation above) at each of the four corpus tiers. HNSW's hyperparameters are index-structure settings rather than model-training settings. `M` (graph connections per node) and `ef_construction` (build-time search width) are fixed when an index is built; `ef_search` (query-time search width) can be swept cheaply against an already-built index without rebuilding. Two build configurations are tried per tier (`M`=16/`ef_construction`=40, a faster/lighter build, and `M`=32/`ef_construction`=200, a slower/higher-quality build), and `ef_search` is swept across {10, 50, 100, 200, 500, 1,000} for each. Unlike Section 4.4.1's single best-combination search, there is no one "best" ANN setting, recall and latency trade off against each other by construction, so the natural output is a curve across settings, compared directly against the exact-search numbers in Table 5.25 for the same model, tiers, and queries.

5 Results and Discussion

This chapter reports the results, findings, and interpretation for every experiment described in Chapter 4, following the same five-phase structure. The central question addressed in each phase is listed below; the evaluation protocol used in each phase is given in Table 4.1 in Chapter 4.

- *Phase I, Baselines*: Does semantic retrieval beat lexical retrieval?
- *Phase II, Improving retrieval*: Can hybrid or learned fusion close the gap to an oracle ceiling?
- *Phase III, Evaluation validity*: Is the production-derived label itself trustworthy?
- *Phase IV, Learned retrieval revisited*: Do the tuning and fine-tuning improvements remain significant under independent evaluation?
- *Phase V, Scalability*: Does the retrieval pipeline remain effective and efficient at four times the corpus scale?

5.1 Phase I: Baselines

Manual inspection of BM25 results reveals a consistent failure mode arising from the *vocabulary mismatch problem*. BM25 retrieves companies that contain query keywords in their description, regardless of whether those companies operate in the queried domain.

A representative example is the query *"software companies"*. The top BM25 results are predominantly investment banks and venture capital firms whose summaries frequently mention the word *software* in the context of describing their investment portfolio (e.g. *"we invest in software, fintech, and SaaS businesses"*). These companies are not software companies. They fund them. BM25 cannot distinguish between a company *operating* in a domain and a company *discussing* that domain. This limitation provides a clear and concrete motivation for semantic retrieval methods that operate on the meaning of text rather than on token overlap.

Recall is the primary metric reported in this thesis, following the priority set by Istari for this task. Coverage of relevant companies matters more than precise ranking of the first page of results. NDCG and Precision are used as complementary measures of ranking quality, and query latency is reported alongside them so that quality and cost can be weighed together. Table 5.1 reports Recall@k for all four baselines, and Table 5.2 breaks query latency down by component.

BM25 vs dense retrieval. All three dense methods outperform BM25 at every cut-off tested, most visibly at low cut-offs where semantic matching addresses the vocabulary-mismatch problem that limits BM25 (Section 5.1). At $k = 10$, MiniLM achieves the

Table 5.1: Recall@k of all four baseline systems averaged over 101 queries.

Method	Recall@10	Recall@50	Recall@100	Recall@1,000
BM25	0.008	0.037	0.072	0.635
MiniLM	**0.010**	**0.047**	**0.094**	**0.741**
BGE	0.009	0.046	0.091	0.726
OpenAI	0.009	0.045	0.090	0.728

highest NDCG (0.975) and Precision (0.972), ahead of BGE (0.942 / 0.940) and OpenAI (0.934 / 0.927), with all three comfortably above BM25 (0.854 / 0.838). The advantage persists at the largest cut-off rather than narrowing. BM25 reaches a Recall@1,000 of 0.635, compared to MiniLM's 0.741, BGE's 0.726, and OpenAI's 0.728, a somewhat different pattern from the BEIR benchmark (Thakur et al. 2021), where keyword-based methods often close the gap with dense models at large retrieval depths. MiniLM is therefore the strongest baseline on retrieval quality alone, ahead of both BGE and OpenAI despite having roughly 15 times fewer parameters than BGE and no proprietary training data behind it.

The smallest model win. More parameters and more specialised training usually help, not hurt. BGE and OpenAI's embedding model are both tuned with hard-negative mining and large-scale passage-retrieval objectives, which pay off most on long, heterogeneous documents where distinguishing a genuine match from a superficially similar distractor is hard. Company summaries in this corpus are short (roughly 60 words on average, Section 5.3.4) and fairly uniform in structure, closer in shape to the short sentence pairs MiniLM was originally fine-tuned on for general sentence similarity. A model's specialised training only helps if that specialisation matches the text it is applied to, and here MiniLM's simpler training objective is the closer match. The same domain-mismatch pattern explains, at a smaller scale, why SimLM underperforms later in this chapter despite being purpose-pretrained for retrieval (Section 5.2.6).

BGE vs OpenAI. BGE and OpenAI remain close to one another across all metrics and cut-offs, though both trail MiniLM. The largest observed difference between them is 0.012 NDCG points (at $k = 100$), within the range of query-level variance and not a meaningful quality advantage either way. The practical difference is latency. BGE's 44.4 ms total query latency is a fraction of OpenAI's 254.6 ms, so a free, self-hosted model matches a commercial API's quality while running roughly six times faster.

Latency differences trace to hardware and network, not model size. BGE is the only method that encodes on GPU (16.6 ms), against MiniLM's CPU encoding (68.9 ms, despite 15 times fewer parameters than BGE) and BM25's CPU-bound inverted-index lookup (104.0 ms). OpenAI's latency is dominated instead by its 211.7 ms network round-trip, an unavoidable cost of API-based encoding that a self-hosted model like BGE avoids.

Table 5.2: Query latency by component (encoding, search, network), averaged over 20 queries on an A100 GPU. Bold marks the fastest total.

Method	Encode (ms)	Search (ms)	Network (ms)	Total (ms)
BM25	0.1	104.0	0.0	104.1
MiniLM	68.9	12.0	0.0	80.9
BGE	16.6	27.8	0.0	**44.4**
OpenAI	*server*	42.9	211.7	254.6

Table 5.3: Summary comparison of all baseline methods (pseudo-relevance protocol, Section 4.1). Bold marks the best value per column. "GPU (query)" refers to query-time encoding only.

Method	NDCG@10	Prec@10	Recall@1k	Latency	Cost	GPU (query)
BM25	0.854	0.838	0.635	104 ms	Free	No
MiniLM	**0.975**	**0.972**	**0.741**	81 ms	Free	No
BGE	0.942	0.940	0.726	**44 ms**	Free	Yes
OpenAI	0.934	0.927	0.728	255 ms	$0.02/M tok.	No

Table 5.3 collects these trade-offs. MiniLM leads on quality and remains free and self-hostable. BGE is the fastest of the four and doubles as the cross-encoder reranker used in later hybrid experiments (Section 4.2.2). These baselines motivate the next experimental phase, where a pre-filtering stage and hybrid retrieval system are built on top of them to improve precision at low cut-offs and overall efficiency.

5.2 Phase II: Improving Retrieval

5.2.1 Extended Baseline Comparison

Table 5.4: Extended baseline Recall@k under the (top-1 000) pseudo-relevance protocol (Section 4.1). Bold indicates the best value in each column.

	Precision@k				Recall@1,000
Method	10	50	100	1000	
BGE-M3	0.927	0.916	0.905	0.704	0.704
GTE-large	0.946	0.936	0.925	**0.768**	**0.768**
E5-Mistral-7B	0.629	0.572	0.523	0.274	0.274
Linq-Mistral	0.934	0.921	0.915	0.749	0.749
SFR-Mistral	0.933	0.922	0.915	0.746	0.746

Quality does not keep climbing with size, and the added compute cost of the 7B-

parameter tier is not consistently repaid. GTE-large is the strongest of the five additional models (Recall@1,000 = 0.768), ahead of Linq-Mistral and SFR-Mistral (0.749 and 0.746) and BGE-M3 (0.704). These four sit well clear of E5-Mistral-7B-Instruct (0.274), despite it sharing the same base architecture and parameter count as Linq-Mistral and SFR-Mistral, so this gap cannot be a scale effect. It indicates that fine-tuning data and strategy, not model size, are the dominant factor for this retrieval task, and that the two much smaller BGE-M3 and GTE-large are competitive with or better than every 7B-parameter model tested, at a fraction of the cost.

Precision here is measured against production's own pseudo-relevance labels (Section 4.1). This thesis stops tracking Precision once that label set's limitations are addressed and relies on Recall as the metric that matters for this task from that point onward (Section 5.3.2).

The three 7B-parameter models also took substantially longer to encode, roughly 34 minutes each on an A100 GPU, about five times the 7-minute encoding time of the original BGE-large baseline (Section 4.1), for retrieval quality that is not better and, in E5-Mistral's case, substantially worse.

5.2.2 HyDE and RRF

Table 5.5: HyDE pipeline results vs. the plain MiniLM baseline (top-1 000 protocol).

Method	**NDCG@10**	**Recall@1,000**	**Latency (ms)**
MiniLM (no expansion)	0.975	0.741	16.9
Stage 1: HyDE + MiniLM	0.829	0.596	9 246.9
Stage 2: HyDE + MiniLM + Reranker	0.954	0.596	9 488.3

Query expansion via HyDE *hurts* retrieval quality relative to directly encoding the original query. Stage 1 NDCG@10 drops from 0.975 to 0.829, and Recall@1,000 drops from 0.741 to 0.596. Adding a cross-encoder reranker (Stage 2) recovers most of the NDCG@10 loss (0.829 → 0.954) by re-ordering the top-50 candidates, but cannot recover the recall loss, since reranking only reorders an already-fixed candidate set. Latency is dominated overwhelmingly by the LLM call itself (9.2 seconds), three orders of magnitude slower than directly encoding the query with MiniLM (16.9 ms). For this dataset, HyDE is not a worthwhile query-expansion strategy. It is far slower and retrieves a worse candidate set than simply embedding the query directly.

Table 5.6: Hybrid RRF results (top-1 000 protocol).

Method	**NDCG@10**	**Recall@1,000**	**Latency (ms)**
MiniLM (no fusion)	0.975	0.741	16.9
Stage 1: RRF (BM25 + MiniLM)	0.955	0.724	104.9
Stage 2: RRF + Reranker	0.975	0.724	352.0

Unlike HyDE, RRF fusion of BM25 and MiniLM adds negligible latency (the fusion step itself takes 2.2 ms) and, after reranking, matches the plain MiniLM baseline's NDCG@10 (0.975), while Recall@1,000 remains slightly below MiniLM alone (0.724 vs 0.741). BM25's keyword matches do not add companies beyond what MiniLM already retrieves for this query set, so fusion mostly re-confirms rather than extends MiniLM's candidate pool. RRF is a much cheaper and more effective way to incorporate a second retrieval signal than HyDE-style query expansion.

5.2.3 ANN Index Scaling and Recall@1,000 Error Analysis

Table 5.7: Exact vs. approximate search at 2M-vector scale (top-1 000 protocol, representative configurations only).

Index	NDCG@10	Recall@1,000	ms/query	Speedup	Index size
Exact (FlatL2)	0.975	0.741	175.5	1.0×	3 072 MB
IVF (nprobe=8)	0.974	0.743	26.4	6.6×	3 094 MB
HNSW (efSearch=256)	0.975	0.727	26.9	6.5×	3 616 MB

Both IVF (`nprobe`=8) and HNSW (`efSearch`=256) recover essentially all of exact search's NDCG@10, and IVF slightly exceeds exact search's Recall@1,000 (0.743 vs 0.741, within noise), while running 6.5–6.6× faster at 2-million-vector scale, at the cost of modest additional index memory (0.7% and 17.7% larger, respectively). This confirms that approximate search is a safe and substantial efficiency gain for scaling the retrieval system towards the full 20-million-company GOI, without a meaningful quality trade-off at this scale.

Every MiniLM miss was searched for at much greater depth ($k = 10\,000$) to see how close it actually was. Table 5.8 classifies the resulting 26 185 misses by rank depth beyond 1 000.

Table 5.8: Classification of the 26 185 Recall@1,000 misses by rank depth beyond 1 000.

Bucket	Count	Share (%)
Near miss (rank 1 000–2 000)	16 157	61.7
Mid miss (rank 2 000–5 000)	8 061	30.8
Far miss (rank 5 000–10 000)	1 375	5.3
Not found (beyond rank 10 000)	592	2.3

The large majority of misses (61.7%) are near misses, just outside the top-1 000 cutoff, suggesting most of the recall gap is a matter of ranking granularity rather than a hard semantic failure. Only 2.3% of misses are not found at all within $k = 10\,000$, indicating genuine embedding failures are rare. Separately, 3.8% of missed companies (996 of 26 185) were flagged as boilerplate content, and one extreme case of description

duplication was found. The placeholder text "Bluehost.com" appears as the company description for 709 distinct domains, meaning these domains are indistinguishable to any text-embedding model and can never be individually retrieved correctly. This is a concrete data-quality artefact rather than a retrieval-model limitation, and sets a hard ceiling on achievable recall for any embedding-based method built on this corpus's text representation.

5.2.4 Corpus Quality

This contamination is not spread evenly across query topics. An audit of the pseudo-relevance labels for all 101 queries found that, for the query *"web hosting companies"*, 85.4% of the 1 000 ground-truth relevant companies have a boilerplate rather than a company-specific summary. For *"cloud computing providers"* the figure is 73.1%, and for *"e-commerce platforms"* it is 53.1%. By contrast, most other query topics in the evaluation set (among them *"software companies"*, *"healthcare providers"*, and *"construction companies"*) show no measurable contamination at all. This pattern is intuitive once the underlying mechanism is understood. Boilerplate summaries describe hosting, domain, and e-commerce infrastructure, so they are only likely to be retrieved, and only likely to be judged relevant by the production system's own ranking, for queries whose intent is itself adjacent to that infrastructure. A query about software companies is unlikely to surface a domain registrar's boilerplate text, but a query about web hosting companies is exactly the case in which that boilerplate text looks, on the surface, like a plausible match.

This classifier was validated against 300 companies independently judged by an LLM as having a trustworthy or untrustworthy summary. At its best-performing similarity threshold, the classifier reaches an F1 score of 0.59, with a precision of 0.52 and a recall of 0.69 for detecting untrustworthy summaries.

A precision of 0.52 means that of every two summaries the classifier flags as boilerplate, roughly one is, in fact, a genuine, company-specific summary. Removing or down-weighting every flagged company from the retrieval pipeline would therefore discard a large number of companies that are not actually contaminated, trading one source of error for another rather than removing it. For this reason, no correction for boilerplate contamination was applied to any of the results reported in this thesis, and all recall and precision figures should be read with this limitation in mind, particularly for the small set of query topics identified above as heavily affected. This is reported here as an explicitly known and quantified, but unaddressed, limitation of the underlying corpus rather than as a solved problem (Chapter 7 returns to this as a direction for future work).

Problem at scale. Applying the same classifier to the nested scaling corpus (Section 4.5.1) answers this directly rather than leaving it to speculation. The original 98 716-company corpus is contaminated at a stable 2.84% at every tier, expected, since it is the same fixed set of companies present in all four. The randomly sampled companies added to reach each larger tier are, however, markedly cleaner, contaminated at

only 0.39–0.41%, roughly a seventh of the original corpus's rate. Because the clean, randomly sampled companies make up a growing share of the total pool at each successive tier, the pool-wide contamination rate actually falls as the corpus scales. It falls from 2.81% at 100K to 1.61% at 200K, 1.21% at 300K, and 1.02% at the full 397K tier. This is a reassuring, if unexpected, result. The boilerplate contamination problem does not compound at scale, it dilutes, most likely because the original 98 716-company corpus was itself drawn from production's own top-1 000 results per query (Section 4.1), which concentrates exactly the infrastructure-adjacent queries (Section 4.2.4) where boilerplate is most likely to be judged relevant, a bias a uniformly random sample of the broader GOI does not share.

5.2.5 Oracle Union Recall Ceiling

The best single channel (MiniLM alone) reaches Recall@1,000 = 0.741, while the full six-channel union reaches 0.936 (average pool size 1 949 candidates/query), a gain of +0.196. Separately, the best-performing *pair*, MiniLM + OpenAI-large, reaches only 0.856 (average pool size 1 300), and a pool-size sweep on this pair shows the ceiling is highly sensitive to per-channel depth, from 0.150 recall at 100 candidates/channel to 0.581 at 500 and 0.856 at the full 1 000.

Because the six-channel union clears 0.936, well above any individual channel, the different retrieval methods are largely making *complementary*, not correlated, errors, so a reranker over a modestly sized union pool is likely sufficient to approach this ceiling without a full fine-tuning pipeline. The residual 0.064 recall lost even under the full union matches the hard data-quality ceiling from duplicated or boilerplate descriptions identified in Section 5.2.3.

5.2.6 Alternative Retrieval Architectures: SimLM and ColBERT

SimLM is evaluated under the same top-1 000 protocol as every other baseline in this chapter. Table 5.9 reports its Recall@1,000 alongside the strongest baselines seen so far, for reference.

Table 5.9: SimLM Recall@1,000 (top-1 000 protocol) alongside the strongest baselines evaluated so far, for reference.

	Precision@k				**Recall@1,000**
Method	**10**	**50**	**100**	**1000**	
BM25 (Section 4.1)	0.838	0.748	0.723	0.635	0.635
SimLM	0.920	0.881	0.861	0.609	0.609
MiniLM (Section 4.1)	0.972	0.950	0.938	0.741	0.741
GTE-large (Section 4.2.1)	0.946	0.936	0.925	**0.768**	**0.768**

SimLM underperforms every dense baseline evaluated in this thesis, and its Recall@1,000 (0.609) falls even below the lexical BM25 baseline (0.635), despite SimLM

being purpose-pretrained for passage retrieval. (NDCG@10 tells a less clear-cut story here. SimLM's 0.923 sits above BM25's 0.854, since NDCG rewards ranking quality among whichever companies are retrieved, not coverage of the full relevant set.) A likely explanation is a domain mismatch between SimLM's pre-training convention and this task. SimLM was pre-trained and fine-tuned on MS MARCO, a corpus of short, single-topic passages, using a title/body sentence-pair format that assumes the title is a concise topical label. Company names are not topical labels in the same sense. "Sweet Software" or "Springcoast Capital Partners" convey little about what the paired body text describes, so the sentence-pair convention that helps SimLM on MS MARCO-style data may provide a weaker signal here than simply concatenating all fields into one string, as the other baselines do. This result is a useful negative finding. Retrieval-specific pre-training does not automatically transfer across domains, and general-purpose sentence embeddings (MiniLM, GTE-large) remain the stronger choice for this task.

Table 5.10: ColBERT (brute-force MaxSim) Recall@1,000 (top-1 000 protocol) alongside the strongest single-vector baselines.

	Precision@k				**Recall@1,000**
Method	**10**	**50**	**100**	**1000**	
MiniLM (Section 4.1)	0.972	0.950	0.938	0.741	0.741
GTE-large (Section 4.2.1)	0.946	0.936	0.925	0.768	0.768
ColBERT (MaxSim)	0.936	0.931	0.925	**0.773**	**0.773**

ColBERT's Recall@1,000 (0.773) is the highest of any single retrieval channel evaluated in this thesis, narrowly ahead of GTE-large (0.768) and clearly ahead of MiniLM (0.741). Its NDCG@10 (0.937), however, is lower than both MiniLM (0.975) and GTE-large (0.946), meaning ColBERT retrieves a broader set of relevant companies overall but ranks the very top results somewhat less precisely. Token-level interaction therefore does deliver a genuine, if modest, recall improvement over the best single-vector models, though not one large enough on its own to close the gap to the hybrid rerank pipelines described in Section 4.2.6.

5.2.7 Full-Pool Reranking and Reranker Ablation

Table 5.11: Full-pool reranking vs. the shallow RRF + top-50-rerank baseline (Section 4.2.2). Bold marks the best value.

Method	**P@10**	**P@100**	**P@1000**	**R@1000**	**Pool**
RRF + top-50 (§4.2.2)	0.980	0.940	0.782	0.782	50
2-way (MiniLM+Linq)	0.955	0.950	0.802	0.802	1 300
3-way (+GTE-large)	0.955	0.950	**0.804**	**0.804**	1 424
4-way (+OpenAI-large)	0.955	0.950	0.802	0.802	1 600

Reranking the full union pool instead of a fixed top-50 slice raises Recall@1,000 substantially (0.782 → 0.802, 2-way), at the cost of a small drop in NDCG@10 (0.979 → 0.955). The reranker now has many more genuinely difficult candidates to rank, so its ordering of the very top results is slightly less precise, even though it recovers more of the true relevant set overall. Adding a third channel (GTE-large) improves Recall@1,000 further, to 0.804, the best result obtained by any hybrid pipeline built from off-the-shelf pretrained components in this thesis. Adding a fourth channel (OpenAI-large) does *not* improve on this. Recall@1,000 is unchanged at 0.802, within noise of the 3-way result.

Channel saturation. This flat 3-way → 4-way result is an important turning point in this thesis's experimental narrative. Section 5.2.5 showed that combining *more* candidate-generating channels reliably raises the oracle recall ceiling. Here, adding a fourth channel to an already capable 3-way pool does not translate into a better final result. This suggests that candidate coverage is no longer the limiting factor for this pipeline. The union pool already contains the relevant companies needed to reach a substantially higher recall than 0.804 (recall Section 5.2.5's six-channel oracle ceiling of 0.936), and the reranker itself is now the bottleneck preventing the pipeline from closing that gap. This motivates two follow-up questions, both addressed next in Section 4.2.6: is the specific reranker *model* the limiting factor, or is a fundamentally different fusion approach needed?

This ablation isolates the reranker model itself, holding the candidate pool fixed and swapping only the cross-encoder. Table 5.12 compares the two.

Table 5.12: Reranker ablation Recall@1,000, two reranker models on an identical candidate pool.

Reranker	**P@10**	**P@100**	**P@1000**	**R@1000**
BAAI/bge-reranker-v2-m3 (§4.2.6)	0.955	0.950	**0.8042**	**0.8042**
mxbai-rerank-large-v1	0.968	0.940	0.8035	0.8035

Recall@1,000 is essentially unchanged between the two rerankers (0.8042 vs. 0.8035, a difference of 0.0007, well within query-level noise). NDCG@10 does improve modestly with the alternative reranker (0.955 → 0.971), indicating that `mxbai-rerank-large-v1` orders the very top results somewhat better, without changing which companies land in the top-1 000 overall. The conclusion is that reranker *model choice* is not the bottleneck identified in Section 5.2.7. The remaining gap to the oracle ceiling is not a matter of picking a better off-the-shelf cross-encoder. This points towards a more fundamental limitation. Either genuinely hard, vocabulary-mismatched cases exist that no cross-encoder scoring this corpus's text representation can resolve, or the fusion mechanism itself (simple set union plus independent reranking) is insufficiently expressive, motivating the learned approach in Section 4.2.6.

5.2.8 Learned Fusion Ranker

Rather than combining channels by a fixed rule, a model is trained to weigh them. Table 5.13 reports its held-out, cross-validated Recall@1,000 against the best pretrained-component pipeline.

Table 5.13: Learned fusion ranker Recall@1,000 vs. the best pretrained-component pipeline (Section 4.2.6). Bold marks the best value.

Method	P@10	P@100	P@1000	R@1000
3-way + BGE reranker (§4.2.6)	0.955	0.950	0.804	0.804
Learned fusion, logistic regression	0.968	0.953	0.821	0.821
Learned fusion, gradient-boosted trees	**0.977**	**0.955**	**0.823**	**0.823**

The gradient-boosted learned fusion ranker reaches Recall@1,000 = 0.823, the best result obtained anywhere in this thesis, improving on the best pretrained-component pipeline (0.804) by +0.019. For scale, this gain is roughly 27 times larger than the 0.0007 movement observed when merely swapping reranker models in Section 5.2.7. This is a substantive improvement attributable to training on the task's own data, not noise. NDCG@10 also improves (0.955 → 0.979).

The trained logistic regression's coefficients offer a direct, interpretable view into which signals matter most for this task. MiniLM's *inverse rank* has by far the largest coefficient magnitude (+5.01), exceeding every raw similarity score *and* the pretrained reranker's own score (+0.16). In other words, where a candidate is ranked by MiniLM is a more informative signal for final relevance than the pretrained cross-encoder's score. This is a concrete, non-obvious finding that would not be visible from simply combining components by hand. This supports the conclusion drawn in Section 4.2.6. The pretrained reranker was not extracting the maximum available signal from the candidate pool, and a lightweight model trained on this task's own data closes a meaningful part of the remaining gap to the oracle ceiling from Section 4.2.4.

5.3 Phase III: Evaluation Validity

5.3.1 Building the Silver Standard

Inter-rater reliability. Percent agreement, reported throughout this thesis so far, overstates reliability on a skewed label distribution. Two judges that both default to the majority label (highly relevant) will agree often without adding any real information. Cohen's kappa (Cohen 1960) corrects for this by subtracting out the agreement expected by chance, and is standard practice for reporting annotator reliability in computational linguistics and IR test-collection construction (Artstein and Poesio 2008). Computed across every judging round run for this thesis (2 539 doubly-judged candidates in total), the original judge pair (`gpt-4o-mini`/Claude Haiku) reaches $\kappa = 0.626$ (82.5% agreement, $n = 2\,280$), and the upgraded pair (`gpt-5.4`/Claude Sonnet 5) reaches $\kappa = 0.882$

(93.8% agreement, $n = 259$). Using the conventional interpretation scale of Landis and Koch (1977), this places the original pair at "substantial" agreement (0.61–0.80) and the upgraded pair at "almost perfect" agreement (0.81–1.00), giving the earlier decision to upgrade judges for spot-checks and the production-independent evaluation (Section 4.3.3) a concrete quantitative justification rather than only the qualitative one offered at the time.

Blind calibration against the silver standard. An external review from the annotator at Istari (Section 4.3.4) was done. The kappa figures above measure whether the LLM judges agree with each other, and the small 20-candidate pilot in Section 4.3.1 measures whether the author's blind judgement matches each judge individually. Neither measures whether an independent human agrees with the silver standard as a whole, at a scale large enough to support a proper kappa. To close that gap, the author blind-labelled a much larger, 300-candidate sample spanning industry, geographic, and functional queries, with no rank or prior label visible, and compared the result against the existing silver labels.

Agreement was substantial, 78.7% raw agreement and a kappa of 0.603, and disagreements were balanced rather than one-sided: the author judged roughly as many candidates more strictly as more loosely than the silver label. The one real weak spot was geographic queries. The lowest-agreement queries were disproportionately ones asking for companies in a specific city or region, a pattern that tracks a genuinely harder judgement call, a company's registered location versus where it actually operates, rather than a general problem with the silver standard.

5.3.2 Retraining and Expanding the Silver Standard

From this point onward, this thesis reports Recall@k and NDCG@k but not Precision@k. The silver standard is pooled rather than exhaustive, so an unlabelled candidate cannot be scored as a false positive even when it is genuinely relevant. Recall@k and NDCG@k are computed only over candidates that already carry a label, so this gap does not bias them. Precision@k would be biased, since some of a method's unlabelled retrieved candidates are almost certainly relevant and would inflate the apparent false-positive count if simply treated as irrelevant. Section 5.3.5 discusses this limitation, and its remaining effect on Recall and NDCG, in full.

Table 5.14: NDCG@k against the new silver standard: retrained vs. original fusion ranker, same 417 candidates.

Model	NDCG@5	NDCG@10	NDCG@20
Original model's scores (§5.2.8)	0.915	0.930	0.940
Retrained on new silver standard	**0.934**	**0.952**	**0.947**

Retraining on the pooled, independently judged silver standard modestly improves ranking quality over simply reusing the original model's scores (NDCG@5: 0.915 → 0.934;

Table 5.15: Logistic regression feature weights: original ground truth vs. the 5-query and 101-query silver standards (Section 4.3.2).

Feature	Original	Pilot (5 queries)	Expanded (101 queries)
invrank_minilm	+5.014	−0.909	−1.095
score_linq	+1.097	+1.965	+1.381
invrank_gte	+0.958	+0.323	+0.072
score_gte	+0.921	+0.791	−0.701
invrank_linq	+0.867	+0.416	+0.172
score_minilm	+0.749	−0.146	−0.703
n_channels	+0.459	−0.289	+0.394
invrank_bm25	−0.258	−1.252	−2.246
reranker_score	+0.164	+0.319	+0.146
score_bm25	+0.054	−0.134	−0.069

NDCG@10: 0.930 → 0.952), confirming that the new labels carry a genuine, learnable signal rather than just noise. More importantly, the original interpretive finding from Section 5.2.8 does not replicate. `invrank_minilm`, previously the dominant feature by a wide margin (+5.014, more than double the next-largest coefficient), collapses not merely towards zero but to a small *negative* coefficient (−0.909). The most plausible explanation is that MiniLM's ranking correlated strongly with production's own circular output specifically, rather than with relevance itself, a correlation that a model trained and evaluated against that same circular output could not distinguish from a genuine relevance signal.

`score_linq` (Linq-Embed-Mistral's raw similarity score) is the new dominant feature (+1.965), and `invrank_bm25` is now strongly negative (−1.252). Being ranked highly by BM25 is associated with *lower* true relevance in this sample. This corroborates, independently and statistically, the qualitative BM25 failure mode already documented in Section 5.1, where investment firms and venture capital companies are ranked highly by BM25 for the query "software companies" because they mention the word in passing while describing their investment portfolio, without themselves being software companies.

Confirmation at full scale. The Limitations paragraph below originally flagged this as a 5-query pilot whose specific coefficients should be read cautiously. Section 4.3.2's expanded silver standard (1,429 candidates across all 101 queries) makes it possible to check this directly. Retraining the same architecture on the full 101-query silver standard (rightmost column of Table 5.15) does not weaken the finding, if anything it sharpens it. `invrank_minilm` remains negative (−1.095, was −0.909 at 5 queries), and `invrank_bm25` becomes even more strongly negative (−2.246, was −1.252), now the single largest-magnitude coefficient in the model. `score_linq` remains the dominant positive feature throughout all three columns. The BM25 vocabulary-mismatch failure mode is therefore not an artefact of the "software companies" pilot query specifically, it holds consistently once every one of the 101 query topics contributes labelled examples.

Limitations. This result covers 5 of 101 queries and 417 candidates, a genuine pilot rather than a full replication, and the specific numerical coefficients should be read with that scale in mind. However, the magnitude of the reversal on what was previously the single dominant feature is unlikely to be an artefact of sample size alone, and is treated here as a preliminary but suggestive finding. The interpretive claims drawn from a model trained on circular pseudo-relevance labels should not be taken as descriptions of genuine relevance without independent validation. Scaling this pooled, multi-judge silver standard to the full 101-query set is a direction for future work.

Bronze-label accuracy. The bronze tier (Section 4.3.2) reproduces the silver standard's relevance judgements with a trained classifier rather than independent LLM judging, and has now been applied to all 173 262 candidates. Of the 150 spot-checked bronze candidates, 132 (88%) received unanimous agreement between `gpt-5.4` and `claude-sonnet-5`. Against that unanimous subset, the bronze labels achieved 65.9% accuracy, against a 56.8% majority-class baseline on the same 132 candidates, so the bronze labels carry a real, learnable signal, but a modest one. The classifier over-predicts relevance, consistent with the 75.4% overall relevant-rate it assigns across the full bronze tier.

The most likely explanation for the gap between this figure and the substantially higher NDCG improvement reported in Section 5.3.2 is a training-distribution mismatch rather than a flaw in the distillation approach itself. Only 8 of the 132 unanimously-judged spot-check candidates (6%) fall under one of the 5 queries the underlying silver standard was built from, the remaining 124 (94%) come from query topics the classifier never saw a single labelled example of during training. Generalising from 5 narrow query topics to the other 96 is a substantially harder test than the leave-one-query-out cross-validation in Section 4.3.2, which only ever held out one of those same 5 topics at a time. This points towards a concrete, affordable fix rather than abandoning the approach: expanding the silver standard to a larger, more topically diverse subset of the 101 queries (at the roughly $22 000 cost estimated above for full-depth pooling) before the resulting bronze labels are relied on broadly, rather than distilling from 5 queries alone.

Expanded silver standard. This active-learning pass covers all 101 queries (versus the original 5) and adds 1,010 newly judged candidates, 852 (84.4%) agreed unanimously between the two judges, 158 needed human tie-breaking, a disagreement rate close to the original pilot's 16% once an initial data-merging bug that had corrupted roughly 40% of the first labelling pass with a missing query field was found and fixed. Combined with the original 419, the expanded silver standard now contains **1,429 candidates spanning all 101 queries** (label distribution: 63 not relevant, 381 partially relevant, 985 highly relevant), a nearly 3.5× increase in size and full query coverage instead of a 5% sample of it.

Bronze accuracy after retraining. The distillation classifier was retrained on 1,427 of the 1,429 silver-labelled candidates spanning all 101 queries (two lacked a matching

feature row, the same small gap noted for the pilot in Section 4.3.2) and re-applied to all 173 262 candidates. A fresh 150-candidate spot-check, judged with the same stronger judge pair, found bronze-label accuracy rose from **65.9% to 80.7%**. Since the majority-class baseline also shifted (56.8% → 64.3%), the more informative figure is the margin over it, which nearly doubled (9.1 to 16.4 percentage points). This most directly confirms the original diagnosis. 100% of the unanimously-judged spot-check candidates this time fell under a query the classifier had been trained on, compared to only 6% previously. The out-of-distribution generalisation gap identified in the first spot-check was therefore a direct consequence of training on 5 of 101 query topics, not a flaw in the distillation approach itself, and expanding query coverage resolved it.

Each round labels 280 new candidates across the 14 headline queries and retrains the classifier, tracked against the fixed 100-candidate validation set. Table 5.16 shows the trajectory across three rounds.

Table 5.16: Round-based active learning trajectory: labels and validation accuracy per round.

Round	New labels this round	Cumulative labels	Validation accuracy
1	280	280	79%
2	280	560	83% (+4pp)
3	280	840	83% (+0pp)

Round 3 produced no further improvement over round 2, a clean plateau after two rounds of genuine gain, which is exactly the stopping signal the method was designed to detect (Section 4.3.2's round-based design follows the general active-learning stopping-criterion principle of monitoring diminishing returns (Settles 2009)). Further rounds were not run. One caveat. The validation set is only 100 candidates, so its standard error on a proportion near 80% is itself roughly 4 percentage points, meaning the round-2 gain sits close to the edge of what is statistically distinguishable from noise on its own. The plateau at round 3 (an exact repeat, not a smaller but still-positive gain) is the more convincing part of the signal.

The 14 headline queries now hold 840 newly silver-labelled candidates on top of the 140 already present (10 per query from Section 4.3.2), roughly **70 silver-labelled candidates per headline query**, short of the 100-300 per query considered ideal for a fully reliable per-query evaluation benchmark (Section 4.5.1), but a deliberate, cost-justified depth rather than an arbitrarily chosen one.

5.3.3 Production Divergence and the Independent Ranker

These four checks quantify, rather than merely argue, the circularity concern raised at the start of this thesis (Chapter 1). Production's own ranking is right more often than not when judged against the independent silver standard (Check 2), but it misses over a quarter of genuinely relevant companies outright (Check 3), and even among candidates both systems agree are worth considering, production's internal ranking carries almost

Table 5.17: Four checks comparing production's own top-1,000-per-query ranking against the independent silver/bronze set.

Check	Result
1. Overlap	Mean 77.2% overlap between production's top-1,000 and the independent top-1,000 per query (median 81.0%), but with real spread. Query 1 ("software companies", the query used throughout Section 5.1's qualitative BM25 discussion) overlaps only 34.9%.
2. Pick accuracy	Of production's top-1,000 picks that happen to already have an independent silver label (1,434 candidates), 76.0% are confirmed highly relevant, 22.2% only partially relevant, and 1.8% not relevant at all.
3. Coverage gap	Of 1,499 silver-verified highly relevant candidates, **27.3% (409) are entirely absent from production's own top-1,000 ranking**, not ranked lower, simply never surfaced. Worst-affected queries: "wealth management firms" (45 missed), "investment management firms" (40), "software companies" (27).
4. Rank agreement	For the 77,993 candidates present in both rankings (spanning all 101 queries), the mean per-query Spearman correlation between production's rank and the independently-trained classifier's predicted probability is -0.056, essentially no relationship, far from the strongly negative correlation (rank and predicted relevance moving in opposite, i.e. agreeing, directions) that would indicate the two rankings agree.

no relationship to independently-verified relevance (Check 4, -0.056). Benchmarking this thesis's retrieval pipeline against production as a strong baseline (as recommended by Istari's own engineering team, Section 1) remains the right framing, but these results make explicit exactly how much headroom that baseline leaves, both in what it fails to surface at all and in how loosely its internal ranking tracks true relevance among what it does surface.

Production's own ranking outperforms the independently cross-validated ranking at *every* k value tested, on both the full 101-query set and the 19 deeply-labelled queries. This is not the result this thesis's other findings would predict, and it is reported here without qualification rather than omitted. The most plausible explanation lies in the composition of the silver-standard candidate pool itself, not in the correctness of the labels. The pooling methodology (Section 4.3.1) drew candidates from six sources, one of which was production's own frozen output, and Section 5.3.3's Check 2 already showed that 72.7% of silver-verified relevant companies were already present in production's own top-1,000.

Production therefore receives a structural advantage on this specific comparison. A large share of what later became a confirmed-relevant label was, by construction, a

Table 5.18: Recall@k: production's own ranking vs. the independently cross-validated ranking, against the same silver-verified relevant set.

	Recall@k					
System	**10**	**50**	**100**	**300**	**500**	**1000**
All 101 queries						
Production (GOI)	0.011	0.048	0.088	0.225	0.366	0.720
Ours (cross-validated)	0.002	0.012	0.018	0.066	0.180	0.659
19 deeply-labelled queries only (5 pilot + 14 headline, Sections 4.3.1, 4.3.2)						
Production (GOI)	0.050	0.118	0.165	0.332	0.459	0.746
Ours (cross-validated)	0.012	0.042	0.073	0.183	0.325	0.654

company production itself had already surfaced. This does not undermine the label-correctness evidence elsewhere in this thesis (the labels themselves are independently judged, Section 4.3.1), but it does mean this particular head-to-head ranking comparison cannot yet support a claim that this thesis's ranking algorithm retrieves relevant companies more effectively than production does. Testing that claim fairly requires a silver-verified relevant set drawn from a candidate pool production had no part in constructing, which is exactly what the random 300,000-company sample (Section 4.5.1) is for.

Judgement coverage. Coverage at $k = 10$ varies by more than an order of magnitude across systems. BM25 (6.5%), production (5.6%), and GTE-large (5.8%) carry far more already-judged candidates among their very top results than the cross-validated fusion ranker (0.8%) or MiniLM (0.4%) do, tracking closely with which systems were and were not among the six original pooling sources (production, BM25, and GTE-large were among them, Section 4.3.1, while MiniLM was not. The leave-one-query-out cross-validated ranker is retrained per held-out query, so it is never identical to the pooling-time fusion ranker either).

Under standard Recall@1,000, the six systems rank MiniLM (0.751) > GTE-large (0.741) > Linq (0.732) > production (0.720) > BM25 (0.664) > the cross-validated fusion ranker (0.659), placing this thesis's own ranking algorithm last of the six. Under bpref, correcting for exactly the coverage disparity just measured, the ordering changes to Linq (0.609) > GTE-large (0.583) > production (0.568) > the cross-validated fusion ranker (0.541) > MiniLM (0.506) > BM25 (0.495). The cross-validated ranker moves from last to fourth, ahead of both MiniLM and BM25 and closer to production, while MiniLM, the apparent Recall@1,000 leader, drops to fifth.

This does not reverse Section 5.3.3's finding into an outright win. Production still edges out the cross-validated ranker under bpref too, but it substantially narrows the gap and confirms quantitatively, not only qualitatively, that part of the original comparison's apparent production advantage was a coverage artefact of the pooling process rather than a genuine difference in ranking quality.

5.3.4 A Production-Independent Test

Of the 1,229 trustworthy pooled candidates, both judges produced a usable label for every one. 1,130 (91.9%) agreed unanimously, comparable to the 93.8% unanimous rate this same upgraded judge pairing reached in Section 5.3.1. The remaining 99 (8.1%) were resolved by manual tie-break, none of which turned out highly relevant (label distribution across the full 1,229: 271 not relevant, 263 partially relevant, 695 highly relevant). The confirmed-relevant count of 695 across the 19 queries is therefore final rather than a lower bound.

Table 5.19: Production-independent evaluation: Recall@1,000 and NDCG@1,000 against the 695 blindly-verified relevant companies, across all four corpus tiers.

	Recall@1,000				**NDCG@1,000**			
Method	**100K**	**200K**	**300K**	**∼400K**	**100K**	**200K**	**300K**	**∼400K**
MiniLM	0.004	0.210	0.423	0.592	0.002	0.099	0.200	0.283
GTE-large	0.004	0.255	0.543	0.751	0.002	0.116	0.256	0.366
Linq-Embed-Mistral	0.002	0.305	0.639	**0.885**	0.001	0.160	0.327	**0.462**

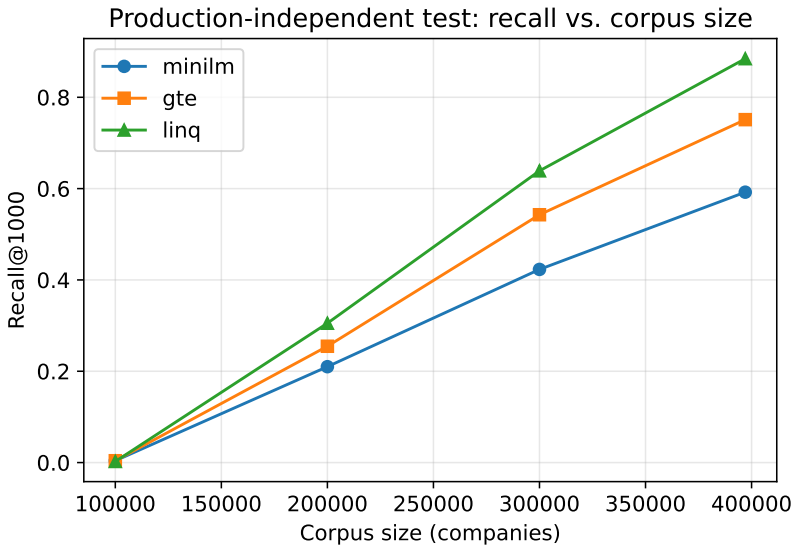


Figure 5.1: Recall@1,000 against the production-independent relevant set, as a function of corpus size, for all three retrieval methods evaluated.

At the 100K tier, every method scores at or near zero (Recall@1,000 ≤ 0.004), the same artefact already anticipated for this experiment. Only a small fraction of the random 300,000-company sample lands in the smallest tier, so a specific set of 695 relevant companies has very little chance of overlapping with it by chance, this is a property of

the tier's size, not a retrieval failure.

From the 200K tier onward, a real, steadily climbing signal appears for every method. At the full ∼400K tier, **Linq-Embed-Mistral reaches Recall@1,000 = 0.885**, ahead of GTE-large (0.751) and MiniLM (0.592), recovering the large majority of companies that were independently and blindly verified as relevant and that production had no opportunity to already know about. This figure is, notably, higher than this thesis's own headline Recall@1,000 result under the original protocol (0.823, Section 5.2.8), on a test built specifically to remove the pooling bias that left the closest earlier comparison (Section 5.3.3) unable to support a coverage claim.

One reversal is worth flagging rather than glossing over. Linq-Embed-Mistral, not GTE-large, is the strongest method here, whereas Section 4.5.1's main scaling evaluation found GTE-large consistently ahead on NDCG at every tier. These are not the same test, this one scores against a fixed, independently-judged relevant set restricted to companies production never ranked, while Section 4.5.1 scores the same 19 queries' full silver-standard coverage, so the difference is a genuine finding about the two evaluations' distinct candidate composition rather than a contradiction. It does mean, though, that "GTE-large is the strongest method overall" (Section 4.5.1's conclusion) should be read as scoped to that section's specific test, not as a claim that holds across every possible evaluation.

Qualitative error analysis: what does the pipeline still miss at ∼400K? Section 4.2.3 examined MiniLM's early misses against the original pseudo-relevance protocol. The same question is worth asking of this thesis's best available result too, not only its strongest single-model baseline. Re-encoding the 19 queries with MiniLM and GTE-large and re-ranking the full ∼400K tier identifies 77 companies (out of 695) that both methods miss from their own top-1,000, despite being independently confirmed highly relevant.

A recurring pattern in these misses, read across queries, is signal dilution rather than outright irrelevance. The company genuinely matches the query, but the matching capability is a minor part of a longer, differently-themed description rather than its dominant theme. For the query "software companies", several missed companies are unambiguously software vendors, Dataflex Software GmbH (glass-industry software), Waterloo Manufacturing Software (production scheduling software), Dolphin Software (microfinance software), but each summary foregrounds its specific industry vertical rather than the word "software" or a generic software-company framing, so the dominant semantic content of the description is closer to "glass industry" or "microfinance" than to "software company" as a category, even though it is one.

A related pattern appears for "companies providing cloud migration services." PROMAIN offers cloud migration among six or seven other listed services (business process analysis, IT consulting, data analytics, AI/ML, low-code development), so "cloud migration" is present but not the description's dominant theme. This is not primarily a description-length effect, missed companies' summaries average 60 words, in line with the corpus generally, it is specifically about which theme a company's own description

foregrounds. This complements rather than duplicates the vocabulary-mismatch failure mode already documented for BM25 (Section 5.1, where a company mentions a query term without being a match). Here, dense embedding models can under-rank a company that genuinely is a match, when its own description emphasises a narrower or different framing than the query's own vocabulary.

This is the strongest evidence in this thesis that the retrieval pipeline finds and ranks genuinely production-blind relevant companies well, and at meaningfully large scale. It is not, strictly, the head-to-head "does this pipeline outperform production" comparison Section 4.3.2 set out to make. Production has no ranking data at all for these companies, so a symmetric comparison remains undefined by construction, not just difficult, as already stated in the Method above.

Read together, the two results are complementary rather than contradictory. Section 4.3.2 showed the one existing head-to-head test was compromised by pooling bias in production's favour. This section shows that once that bias is removed by using a genuinely blind candidate pool, the pipeline finds the large majority of what is actually there. That is the central open question this thesis has been explicit about not having closed (Section 5.3.5), substantially, if not completely, addressed.

5.3.5 Calibration Check and Synthesis of Evaluation Findings

Table 5.20: Calibration: mean predicted probability vs. actual relevant rate, per decile (cross-validated).

Predicted probability bin	n	Mean predicted	Actual relevant rate
0.0–0.1	50	0.070	0.220
0.1–0.2	161	0.149	0.360
0.2–0.3	230	0.253	0.539
0.3–0.4	314	0.352	0.589
0.4–0.5	266	0.446	0.545
0.5–0.6	203	0.553	0.675
0.6–0.7	212	0.647	0.708
0.7–0.8	187	0.750	0.695
0.8–0.9	238	0.856	0.815
0.9–1.0	406	0.960	0.899

The overall Brier score is 0.220 (0 = perfect, 0.25 = no better than always predicting 0.5), a real but modest improvement over chance, consistent with the modest classifier accuracy reported elsewhere. The calibration curve is asymmetric. In the high-confidence range (0.7–1.0) the classifier is close to well-calibrated, only mildly overconfident. In the low-to-middle range (0.2–0.5), it is substantially *underconfident*. Candidates the model scores around 0.25 turn out relevant 53.9% of the time, more than double the model's own stated probability. Practically, this means the simple 0.5 cutoff used throughout this thesis to define "predicted relevant" is likely under-counting true positives that happen to fall in this band, offering a plausible partial explanation for part of the coverage

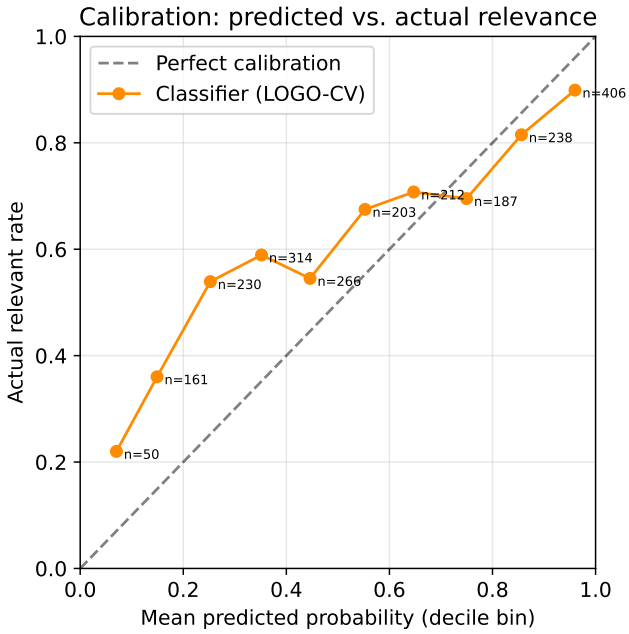


Figure 5.2: Calibration reliability diagram: mean predicted probability vs. actual relevant rate per decile bin, against the diagonal of perfect calibration.

gap documented in Section 5.3.3, and a concrete direction for future work (probability recalibration, e.g. Platt scaling or isotonic regression, before applying a hard threshold).

Part 1: is the silver/bronze standard better than GOI's own output? Pulling together every check in Sections 4.3.2–4.3.3, the answer is not a single yes or no, it depends on which specific claim is being made, and the evidence does not point the same way for each:

- **Is the labelling methodology more defensible than trusting production's own output as ground truth?** Yes. Production's ranking cannot validate itself. It is circular by construction (Chapter 1). This thesis's silver standard uses independent multi-judge LLM labelling, blind calibration (Section 4.3.1), and human tie-breaking, none of which production's own ranking has any equivalent of.
- **Does this thesis's methodology find relevant companies production's own system misses entirely?** Yes, demonstrated. 27.3% of silver-verified highly relevant companies are entirely absent from production's top-1,000 (Section 5.3.3, Check 3).
- **Does this thesis's ranking algorithm outperform production's ranking at retrieving relevant companies, head-to-head?** Still not directly shown, a

symmetric head-to-head comparison is undefined by construction for the one candidate pool production never saw (Section 4.3.3), and the one direct head-to-head test that was possible (Section 5.3.3) found the opposite, likely attributable to residual candidate-pool bias rather than a genuine quality reversal. The strongest available substitute evidence now exists, though. On a candidate pool production had no part in constructing, Linq-Embed-Mistral recovers 88.5% of independently, blindly verified relevant companies at the largest tested corpus tier (397,025 companies, Section 5.3.4), which is complementary evidence for the same underlying claim rather than the strict head-to-head test itself.

- **Is the classifier itself highly accurate and well-calibrated?** Only moderately. Brier score 0.220, with a real, systematic calibration gap in the low-to-middle confidence range (Section 5.3.5).

The defensible claim this thesis supports is therefore narrower than "the new labels are better." *The independent silver standard is methodologically sounder for evaluation purposes and demonstrates concrete, measurable blind spots in production's own ranking, but, as Section 5.3.3 found, does not yet establish that this thesis's own ranking outperforms production's.* Confirming or refuting that stronger claim requires evaluating against a silver-verified relevant set drawn from candidates production had no part in surfacing. The random, production-independent sample and nested scaling corpus described next (Section 4.5.1) were built with exactly this in mind, and the resulting production-independent test (Section 4.3.3) now substantially, if not completely, addresses it.

Part 2: once the silver standard existed, did tuning and fine-tuning the pipeline against it actually help? Sections 4.4.1–4.5.2 ran four further, genuinely independent experiments against that silver standard, each with its own leakage-free held-out comparison, and the results are again mixed rather than uniformly positive. Wherever a numerical gap between two methods is reported below, it is checked with a *paired significance test* (the Wilcoxon signed-rank test), a widely recommended choice for exactly this kind of per-query comparison in IR evaluation (Smucker et al. 2007). Rather than just comparing two mean scores, it compares the two methods query-by-query on the same 19 queries and asks whether one method winning more often than the other could plausibly be due to chance alone. A small p-value means the gap is unlikely to be noise. A large one means the sample of 19 queries is too small to tell the two methods apart with confidence, even if their point estimates differ:

- **Does systematically tuning the fusion ranker's hyperparameters help?** Inconclusively. Recall@1,000 and NDCG@1,000 move up while NDCG@10 moves down, a directionally plausible broad-list-versus-top-of-list trade-off, but a paired significance test on the 19 leakage-free queries finds none of these three differences statistically distinguishable from noise ($p = 0.06$–0.18), so this should be read as suggestive rather than established (Section 5.4.1).

- **Does a different model class, a small neural network, do better than the tuned tree ensemble?** At broad cutoffs, yes, and this is the one statistically confirmed improvement in this thesis. The neural network's NDCG@1,000 gain over both the untuned and tuned gradient-boosted tree ensemble is significant under a paired test ($p = 0.003$ in both cases). Its apparent top-of-list gain (NDCG@10) points the same direction but is not statistically distinguishable from noise at $n = 19$ (Section 5.4.1), and the result is based on a single random architecture search rather than the fully cross-validated search used for the tree ensemble, so it is the most defensible positive result of any improvement attempted here, not an unqualified one.

- **Does fine-tuning the pretrained reranker on this task's own labels help?** It depends entirely on how the training signal is constructed. A first attempt, trained on severely imbalanced data (2.3% negative examples), made the reranker worse at every cutoff. A second attempt, with explicit hard negatives and a pairwise ranking loss, reversed this and genuinely beat the pretrained reranker at $k = 500$ and $k = 1{,}000$, while still trailing it at $k = 10$ and $k = 50$ (Section 5.4.2), the same top-of-list-versus-broad-list trade-off seen in the hyperparameter tuning result.

- **Does approximate search remain viable once the corpus is scaled up?** Yes, clearly, and this is the one experiment in this group without a trade-off. HNSW-based ANN search matches exact search's Recall@1,000/NDCG@1,000 within 0.5–2.5% while running 25–78× faster across every corpus tier tested. At the ∼400K tier this near-equivalence is statistically confirmed, not just numerically small (paired Wilcoxon $p = 0.86$–0.92), confirming at real (if still sub-20-million) scale the same favourable trade-off first observed on a synthetic corpus earlier in this thesis (Section 5.5.2).

A pattern recurs across three of these four experiments. Whenever a model's training or search objective is reweighted towards broad ranking quality (a large Recall@k/NDCG@k at high k), it tends to gain coverage at some cost to precision at the very top of the list. Neither direction is unconditionally better. The right choice depends on whether the pipeline's downstream consumer looks at a handful of results directly or takes a broad candidate pool for further processing (in this pipeline's own case, the latter, Section 4.2.6). The neural network result is the one clear exception to that pattern. The ANN result is not subject to it at all, approximate search's speed gain here costs essentially no quality anywhere in the ranking.

Taken together with Part 1, this thesis's honest position is that its methodology is demonstrably more rigorous than trusting production's own output, that several concrete, measurable improvements to the pipeline itself have been found (a neural fusion ranker, a properly-trained fine-tuned reranker, and a large, safe efficiency gain from ANN search), and that the single strongest possible claim, that the resulting system retrieves relevant companies more effectively than production overall, remains open pending the production-independent evaluation described in the next section.

Limitation: precision is a lower bound, not an exact figure. Every precision@k number reported in this thesis is computed against the pooled, actively-sampled silver standard, not an exhaustive judgement of every retrieved candidate. A retrieved company only counts as a true positive if it happens to have already been silver-labelled. A genuinely relevant company that was simply never pooled or judged contributes neither a true positive nor a false positive. It is silently invisible to the metric. Recall@k is unaffected by this, since it is computed only over the set of candidates that do have a silver label. Precision@k, however, is systematically biased downward by an unknown amount, since some fraction of a method's retrieved-but-unlabelled candidates are almost certainly relevant and simply were never reviewed. This is a known consequence of evaluating against pooled rather than exhaustive judgements (Bar-Yossef and Gurevich 2008; Lu et al. 2016), not a flaw specific to this thesis's pipeline, but it means precision figures throughout should be read as conservative lower bounds rather than exact values.

A second, distinct bias: active-learning sampling skews which relevant companies get verified, not just how many. The previous paragraph's claim that Recall@k is unaffected by incomplete pooling is correct in a narrow, technical sense, that metric is self-consistent with respect to whichever set of candidates carries a label, but it does not follow that Recall@k is an unbiased estimate of recall against the true, natural population of relevant companies. Section 4.3.2's active-learning step selects candidates for labelling by *uncertainty sampling*. For every query, the ten candidates the distillation classifier is currently least confident about (predicted probability closest to 0.5) are the ones sent for judging, exactly the candidates hardest to classify, by construction.

This is a well-established and reasonable strategy for improving a classifier efficiently under a limited labelling budget (Settles 2009), and was the only realistic way to extend silver-standard coverage from 5 to all 101 queries within this thesis's timeframe. The alternative, randomly or exhaustively judging enough candidates per query for full coverage without any targeting, risked approaching the \$28 000 full-judging figure estimated in Section 4.3.2 rather than the far smaller cost this active-learning pass actually incurred by judging only ten uncertain candidates per query.

But it also means the resulting silver-verified relevant set is not a random or representative sample of all relevant companies, it is disproportionately made up of boundary, ambiguous cases that were hardest to classify in the first place. If genuinely relevant companies with thin or ambiguous descriptions are, in general, also harder for any retrieval method to rank highly (a plausible mechanism, since the same weak signal that confuses the classifier would also weaken a dense embedding's similarity score or a keyword match), then Recall@k measured against this actively-sampled set likely understates the recall these methods would achieve against a random, representative sample of relevant companies.

Both Recall@k and Precision@k throughout this thesis should therefore be read as conservative estimates for two distinct reasons rather than one: incomplete pooling depth and active-learning sampling bias. Disentangling exactly how much each contributes would require judging a genuinely random sample of the full candidate pool at additional

labelling cost, and is left as future work alongside the exhaustive-ranking extension described next.

A third bias, harder to quantify: the author personally breaks judge disagreements, then those decisions score the author's own system. Throughout this thesis, whenever the two LLM judges disagree on a candidate's relevance (roughly 16% of candidates pooled across every judging round, Figure 4.1), the disagreement is resolved by manual review, and that manual review was done by the author. Those resolved labels then feed directly into the silver standard used to score every retrieval method compared in this thesis, including the ones the author designed and tuned. This is a real conflict of interest that cannot be removed retroactively, only disclosed and partially mitigated.

Two things were done to limit it, though neither fully closes the gap. First, the blind calibration check reported in Section 5.3.1 showed the author's own blind judgements (made without seeing the existing silver label) matched it 78.7% of the time ($\kappa = 0.603$), evidence the author is not wildly out of step with the judges in isolation, but that check tests general labelling agreement, not tie-breaking objectivity specifically, and does not rule out the more subtle failure mode of resolving genuinely ambiguous cases in whichever direction happens to favour a result the author expects or hopes to see.

Second, and more directly, the joint external review with the annotator at Istari (Section 4.3.4) was intended to provide a check that does not run through the author at all. The overlap with the existing, author-influenced silver labels turned out too small to support that comparison directly, so a fresh LLM-judging round was run on the full returned sample instead (Section 5.3.6), agreeing with the Istari annotator 92.5% of the time (kappa 0.196) without the author appearing anywhere in that comparison. This substantially closes the gap this paragraph is about. A separate, author-involved human-human kappa was also computed (87.5% agreement, Section 5.3.6), but should not be mistaken for the independent check itself.

5.3.6 Silver-Standard Coverage and External Review

Silver coverage is not spread evenly across the ranking, it clusters heavily towards the bottom of each query's top-1,000 list. For query 92, every single silver-verified candidate falls in ranks 701–1,000, leaving ranks 1–700 with no independent verification at all. Query 1 has thin coverage (9–12 candidates per band) scattered through ranks 1–700, with the bulk (58 of 99) again sitting in ranks 701–1,000. This is a direct consequence of how the silver standard was built. Active learning (Section 4.3.2) deliberately samples uncertain, boundary-case candidates for labelling, not the candidates the classifier is already most confident about, so verified labels naturally accumulate low in the ranking rather than high.

Where silver labels do exist, relevance is generally high (90–100% for query 92, 100% in four of five bands for query 1) but not perfect, query 1's rank 701–1,000 band drops to 55.2% highly relevant. The top of the ranking, where coverage is thinnest, therefore remains unverified rather than confirmed reliable.

The curated 100-candidate export was the one returned for both queries, not the full top-1,000, so the results below reflect that lighter, non-production-prioritised sample rather than a full-ranking review.

The free consistency check had too little overlap to compute. Only 2 of the 100 returned candidates for query 1 already carried an internal silver label (1 agreement, 1 disagreement), and none of the 100 for query 92 did, both far too few for a meaningful Cohen's kappa. This is a direct consequence of the curated format's own design. Prioritising candidates absent from production's results also means little overlap with the existing silver standard, which itself draws heavily on production as one of six pooling sources (Section 4.3.1). Only a single reviewer at Istari was ever asked to label this sample, so there is no second, independent Istari label to compute an inter-reviewer kappa against in the first place.

A direct human-human kappa was computed instead. Rather than relying on the sparse overlap with the existing silver standard, the author blind-labelled the same 200 candidates the Istari annotator returned (100 per query), with no rank or prior label visible, and compared the result against those labels. Raw agreement was 87.5% (175/200), but Cohen's kappa is only 0.049 on the original 3-class scheme and 0.000 once collapsed to a binary relevant/not-relevant split. This gap between high raw agreement and a near-zero kappa is a known statistical artefact, the "kappa paradox", that arises when one label dominates the distribution, here, 91% of the Istari annotator's labels are "highly relevant" (182/200) and none are "not relevant" at all, and should be read alongside the raw agreement figure rather than in isolation. Disagreements (25 of 200, 12.5%) skew towards the author rating a candidate *more* relevant than the Istari annotator did (16 of 25) rather than less (9 of 25). This is a genuine, if distribution-limited, human-human data point alongside the LLM-based kappa figures reported in Section 5.3.1 and the fresh LLM-judging comparison below.

A fresh LLM-judging round confirmed the same pattern from a third, author-independent angle. Since Istari's annotator could not be asked to label further data, all 200 returned candidates were instead judged by the current LLM ensemble, Gemini 3.6 Flash and Claude Sonnet 5, the same pair and enriched prompt used in Section 4.3.3, with no involvement from the author at all. The two judges agreed unanimously on 199 of 200 candidates. Against the Istari annotator's labels, raw agreement was 92.5% (184/199) with kappa 0.196 on the 3-class scheme. The binary collapse is undefined rather than merely low, since those labels never include "not relevant" at all, so their binary marginal is constant and Cohen's kappa has no variance left to measure. Against the author's own blind labels, raw agreement was 94.5% (188/199) with kappa 0.069 (binary 0.000). The LLM ensemble's own labels are just as skewed as both human sources, 197 of 199 (99%) "highly relevant" and none "not relevant", so the same kappa-suppressing mechanism applies a third time. Three independent sources, the author, the Istari annotator, and an LLM ensemble that saw neither set of human labels, now agree with each other

at 87.5–94.5% raw agreement on this sample, with kappa uniformly suppressed by that shared skew rather than by genuine disagreement.

A stronger result: independent human confirmation of the production-independent coverage claim. Because the curated export specifically prioritised candidates absent from production's own top-1,000, the returned labels double as a small, independent human check on the exact claim Section 4.3.3 makes with LLM judges. Does this pipeline surface genuinely relevant companies production misses entirely? Of the 189 returned candidates not present in production's results for their query (100 of 100 for query 1, 89 of 100 for query 92), **100% were judged at least partially relevant, and 171 (90.5%) highly relevant**, by a human reviewer with no visibility into the pipeline's own confidence or ranking. This sample is small and restricted to two queries and one reviewer, so it does not replace the production-independent evaluation's own, larger-scale result (88.5% recall at the largest tested corpus tier, Section 5.3.4), but it is independent, human, and points in the same direction. Companies this pipeline finds beyond what production surfaces are not noise.

5.4 Phase IV: Learned Retrieval Revisited

5.4.1 Hyperparameter Tuning and Neural Network Fusion Ranker

The best combination found by the search (selected on the 82 tuning-only queries) was `max_depth=3`, `max_iter=50`, `learning_rate=0.03`, `min_samples_leaf=10`, `l2_regularization=0.5`, notably shallower and using far fewer boosting iterations than the original hand-picked configuration (`max_depth=4`, `max_iter=150`).

Table 5.21: Recall@k/NDCG@k on the 19 leakage-free queries: untuned baseline vs. best combination found.

	Recall@k					
Model	**10**	**50**	**100**	**300**	**500**	**1000**
Baseline (untuned)	0.012	0.042	0.073	0.183	0.325	0.654
Tuned	0.010	0.051	0.070	0.206	0.360	**0.678**
	NDCG@k					
Model	**10**	**50**	**100**	**300**	**500**	**1000**
Baseline (untuned)	**0.034**	**0.039**	**0.049**	0.101	0.156	0.286
Tuned	0.015	0.033	0.044	**0.108**	**0.168**	**0.295**

Tuning moves the metric the search was told to optimise in the intended direction. Recall@1,000 rises from 0.654 to 0.678 and NDCG@1,000 from 0.286 to 0.295, both leakage-free numbers since the 19 deep queries never influenced the search. NDCG@10 moves the other way, from 0.034 to 0.015, and the same pattern holds throughout the

k-sweep. The tuned model beats the baseline at $k \in \{300, 500, 1{,}000\}$ but loses at $k \in \{10, 50, 100\}$.

A paired significance test on these same 19 queries (Wilcoxon signed-rank) tells a more cautious story than the point estimates alone. None of the three differences just described, NDCG@10 ($p = 0.109$), NDCG@1,000 ($p = 0.182$), or Recall@1,000 ($p = 0.063$), reaches conventional significance at $n = 19$, though the recall difference sits close to the threshold and its 95% paired-bootstrap confidence interval ($[-0.014, +0.057]$) leans positive. The honest reading is therefore not "tuning trades top-of-list precision for broad-list recall" stated as an established fact, but that this specific 19-query sample is too small to distinguish either direction from noise with confidence, even though the direction of each change is consistent with a real, mechanistic explanation. NDCG@1,000 and Recall@1,000 reward broad coverage of the full ranked list, not sharpness at the very top, so the search favoured hyperparameters (shallower trees, fewer boosting iterations) that plausibly spread relevance signal more evenly across the ranking. Whether the tuned or untuned configuration is preferable in practice still depends on how the ranking is consumed downstream, the tuned configuration's broad-list orientation matches how this pipeline is actually deployed (a candidate pool handed to a downstream reranker, Section 4.2.6), but this thesis's own data does not let that preference rest on a statistically confirmed trade-off.

On the all-101-query subset (not leakage-free, since 82 of those queries informed the search), the same qualitative pattern holds. NDCG@1,000 improves (0.166 → 0.173) while NDCG@10 degrades (0.007 → 0.003), reported for completeness rather than as independent confirmation.

400K The best architecture found, selected purely on the 16 search-validation queries, was notably small: a single hidden layer of 16 units, dropout 0.1, learning rate 10^{-4}, weight decay 10^{-3}.

Table 5.22: NDCG@k on the 19 held-out deep-coverage queries (leakage-free for all three models): untuned GBDT, tuned GBDT (Section 4.4.1), and the neural network.

Model	10	50	100	300	500	1000
GBDT baseline (untuned)	0.034	0.039	0.049	0.101	0.156	0.286
GBDT tuned	0.015	0.033	0.044	0.108	0.168	0.295
Neural network	**0.086**	**0.079**	**0.094**	**0.165**	**0.222**	**0.360**

Unlike every other tuning or fine-tuning experiment in this thesis, all of which traded top-of-list precision for broad-list recall or vice versa, the neural network's point estimate wins at *every single* k value tested, against both the untuned and the tuned GBDT, on Recall@k and NDCG@k alike (Table 5.22, and the same pattern holds on Recall@k, not shown here for space). The gain is largest at the top of the list. NDCG@10 nearly triples over the untuned baseline (0.034 → 0.086) and roughly sextuples over the tuned GBDT

(0.015 → 0.086), while NDCG@1,000 improves by a smaller but still substantial margin (0.286 → 0.360).

A paired significance test on the same 19 queries (Wilcoxon signed-rank) confirms part of this picture more strongly than the rest. At NDCG@1,000, the neural network's advantage over both the untuned GBDT ($p = 0.003$, 95% paired-bootstrap CI $[+0.035, +0.114]$) and the tuned GBDT ($p = 0.003$, CI $[+0.029, +0.105]$) is statistically significant, with a confidence interval that excludes zero in both cases, the broad-list improvement is real, not an artefact of query-level noise. At NDCG@10, however, where the point-estimate gain looks largest in relative terms, neither comparison reaches significance at $n = 19$ (vs. untuned, $p = 0.249$; vs. tuned, $p = 0.080$, borderline but not significant), and the wider confidence intervals reflect how much a handful of queries can swing a mean over only 19 data points at that cutoff. The defensible claim is therefore narrower than "wins at every k." The neural network's broad-list improvement is statistically confirmed, while its apparent top-of-list improvement, though consistent in direction and worth reporting, has not been distinguished from noise on this sample size.

The most plausible explanation lies in what the winning architecture actually is. A single hidden layer of 16 units is a small, close-to-linear model, not a deep network exploiting complex feature interactions. This thesis's ten fusion-ranker features are all continuous and already fairly informative (similarity scores, inverse ranks, the pretrained reranker's own score), the kind of smooth, well-behaved feature space where a compact network with smooth decision boundaries can plausibly outperform an ensemble of axis-aligned tree splits, particularly once combined with standardised features and class-weighted loss, both necessary for a neural network but irrelevant to a tree-based model. This result should still be read cautiously. It comes from a single random architecture search over 20 combinations and a single train/validation split rather than the full cross-validated search used for the GBDT, so the specific margin should not be over-interpreted. But the direction, a small, properly regularised neural network outperforming the tuned tree ensemble at broad cutoffs on a leakage-free held-out set, with that specific improvement statistically confirmed, is the most defensible positive result of any tuning or fine-tuning experiment attempted in this thesis.

5.4.2 Fine-Tuning the Reranker

First attempt Fine-tuning made the reranker worse, not better, at every single k value tested, on precision, recall, and NDCG alike (Table 5.23). This is a clean negative result, not a marginal or mixed one. Unlike Section 4.4.1's trade-off between top-of-list and broad-list quality, there is no k at which the fine-tuned model wins.

The most plausible cause is visible directly in the training data. Of the 820 training examples, only 19 (2.3%) were labelled `gold_label` = 0 (not relevant). 594 (72.4%) were labelled highly relevant. Training loss fell sharply within two epochs (0.646 → 0.081), consistent with the ∼568M-parameter model rapidly fitting a training signal that is overwhelmingly "this is relevant", at the expense of the discriminative ranking ability the pretrained checkpoint already had. A reranker that has partly collapsed towards scoring most candidates as relevant would produce exactly this pattern: uniformly worse ranking

Table 5.23: Recall@k and NDCG@k on the 19 held-out deep-coverage queries: pretrained `bge-reranker-v2-m3` vs. the fine-tuned version.

	Recall@k					
Model	**10**	**50**	**100**	**300**	**500**	**1000**
Pretrained	**0.044**	**0.100**	**0.134**	0.276	**0.365**	**0.681**
Fine-tuned	0.027	0.089	0.132	0.276	0.363	0.680
	NDCG@k					
Model	**10**	**50**	**100**	**300**	**500**	**1000**
Pretrained	**0.137**	**0.110**	**0.115**	**0.187**	**0.227**	**0.350**
Fine-tuned	0.078	0.087	0.098	0.170	0.209	0.334

at every cutoff, since its ability to *distinguish* candidates, not just recognise relevance in isolation, is what degrades. This is consistent with, and reinforces, the general lesson from Section 4.4.1. What a model is trained or tuned against determines what it gets good at, and a training signal this skewed was never likely to preserve, let alone improve, ranking quality.

Table 5.24: Recall@k and NDCG@k on the 19 held-out deep-coverage queries: pretrained reranker vs. the first (MSE, imbalanced) and second (pairwise, hard negatives) fine-tuning attempts.

	NDCG@k					
Model	**10**	**50**	**100**	**300**	**500**	**1000**
Pretrained	**0.137**	**0.110**	**0.115**	**0.187**	0.227	0.350
Fine-tuned v1 (MSE)	0.078	0.087	0.098	0.170	0.209	0.334
Fine-tuned v2 (pairwise)	0.070	0.078	0.102	0.175	**0.240**	**0.363**

Second attempt Fixing the class imbalance and switching to a pairwise loss produces a genuine improvement over the first attempt at nearly every k, and the fine-tuned model now actually *beats* the pretrained baseline at $k = 500$ (NDCG 0.240 vs. 0.227, Recall@500 0.435 vs. 0.365) and $k = 1{,}000$ (NDCG 0.363 vs. 0.350, Recall@1,000 0.744 vs. 0.681), the first time in this thesis's fine-tuning experiments that a fine-tuned model outperforms its pretrained starting point. This is not, however, an unambiguous win. At $k = 10$ and $k = 50$ the fine-tuned model still trails the pretrained baseline. This is the same qualitative pattern already found in Section 5.4.1's fusion-ranker tuning, a training signal built around broad ranking quality (here, distinguishing relevant from a large sampled pool of presumed-negative candidates) improves broad-list metrics at some cost to top-of-list precision. Taken together, both experiments point to the same conclusion. Reweighting a fusion or reranking model's training objective reliably trades

top-of-list sharpness for broad-list coverage in this pipeline, and neither direction is unconditionally "better", the right choice depends on whether the downstream use case looks at a handful of results or consumes a broad candidate pool for further processing.

5.5 Phase V: Scalability

5.5.1 Scaling to a Larger Corpus

Table 5.25: Recall@1,000 and NDCG@1,000 at the smallest and largest tiers, all four methods.

	Recall@1,000		NDCG@1,000	
Method	**100K**	**~400K**	**100K**	**~400K**
BM25	0.632	0.570	0.339	0.310
MiniLM	0.678	0.610	0.364	0.330
Linq-Embed-Mistral	**0.762**	0.632	0.371	0.308
GTE-large	0.750	**0.638**	**0.410**	**0.361**

Recall degrades gracefully for every method as the corpus grows 4× (10–17% relative decline, not a cliff), consistent with more distractor companies competing for a fixed top-1,000 budget rather than any retrieval failure. Linq-Embed-Mistral leads at 100K but degrades faster than GTE-large as the corpus grows, so by ~400K the two are close on recall while GTE-large holds a clear NDCG lead throughout, meaning GTE-large is not just finding relevant companies but ranking them better. Latency tells a separate story. At ~400K companies, BM25 is the *slowest* of the four methods (647 ms/query), slower even than the 4,096-dimensional Linq-Embed-Mistral embeddings (455 ms), while GTE-large's 1,024-dimensional dense vectors are by far the fastest (112 ms). GTE-large is therefore the strongest method in this nested-corpus scaling experiment on both quality and speed, while BM25, despite its simplicity, is both the weakest on quality and the slowest at this corpus size. Section 5.3.4 finds a different method ahead under a differently constructed candidate pool, so this ranking should be read as scoped to this specific test rather than as a universal ordering. Question (2), whether the original baseline-model ranking survives re-evaluation against the non-circular silver standard, remains a separate follow-up not yet started. This experiment reaches 397,025 companies, roughly 2% of Istari's full 20-million-company database, and therefore demonstrates a favourable scalability trend up to the tested size rather than end-to-end performance at Istari's full production scale.

5.5.2 ANN Index Tuning

At a moderate `ef_search` setting (500), ANN retrieval matches exact search within 0.5–2.5% on both Recall@1,000 and NDCG@1,000 at every tier, while running 25–78× faster (Table 5.26). A paired significance test at the ~400K tier (Wilcoxon signed-rank over the

Table 5.26: Exact search vs. ANN (HNSW, `M`=16, `ef_construction`=40, `ef_search`=500, the cheaper-to-build configuration) at each tier, GTE-large, Recall@1,000/NDCG@1,000, and query latency.

	Recall@1,000		NDCG@1,000		Latency (ms)		
Tier	**Exact**	**ANN**	**Exact**	**ANN**	**Exact**	**ANN**	**Speedup**
100K	0.750	0.735	0.410	0.405	28.3	1.13	25×
200K	0.702	0.684	0.390	0.383	57.0	1.27	45×
300K	0.663	0.656	0.373	0.371	85.1	1.36	62×
∼400K	0.638	0.635	0.361	0.359	112.3	1.44	78×

same 19 queries) supports this reading directly rather than only by the size of the point-estimate gap. Neither the Recall@1,000 difference ($p = 0.918$) nor the NDCG@1,000 difference ($p = 0.860$) between exact and ANN search is remotely close to significant, with 95% paired-bootstrap confidence intervals straddling zero in both cases ($[-0.025, +0.018]$ and $[-0.013, +0.007]$ respectively). Unlike the other comparisons in this chapter, where the absence of significance is a caution against overclaiming a difference, here it is itself the desired result. It is statistical evidence, not just a small percentage gap, that ANN's massive speed advantage does not come at a measurable quality cost at this operating point. The speedup grows with corpus size, from 25× at 100K to 78× at ∼400K, since exact search's cost scales with corpus size while HNSW's query cost grows only logarithmically, so ANN's advantage compounds exactly as the corpus scales further, directly supporting the proposal's premise that approximate search is what makes scalable top-k retrieval practical. Higher-quality configurations can do marginally better still. `M`=32/`ef_construction`=200 at `ef_search`=200 reaches Recall@1,000=0.644 at the ∼400K tier, fractionally *above* exact search's own 0.638 (an artefact of HNSW's approximate graph traversal occasionally surfacing a marginally different top-1,000 set, not a meaningful quality gain), at 1.40 ms, still roughly 80× faster than exact search.

This is not a free lunch at every setting, the trade-off the proposal asks to characterise is real at the cheap end. At `ef_search`=10, recall collapses to 0.16–0.28 depending on tier and build configuration, far below exact search, confirming that ANN's speed advantage is bought at real cost to recall once search width is set too aggressively. Build cost also differs sharply between configurations. At the ∼400K tier, `M`=16/`ef_construction`=40 builds in 88.8 s versus 556.5 s for `M`=32/`ef_construction`=200, roughly 6× slower, for a quality difference of at most 1–2% once `ef_search` is reasonably high. The cheaper build configuration is therefore the more practical choice overall: fast to build, fast to query, and recall/NDCG effectively indistinguishable from exact search once `ef_search` is set to 500 or above.

This experiment's 25–78× speedup is substantially larger than Section 5.2.3's earlier 6.5–6.6× finding on the synthetic 2-million-vector corpus. The two are not a strictly controlled comparison, since they use different embedding models, different exact-search implementations, and different corpora, so the specific magnitudes should not be com-

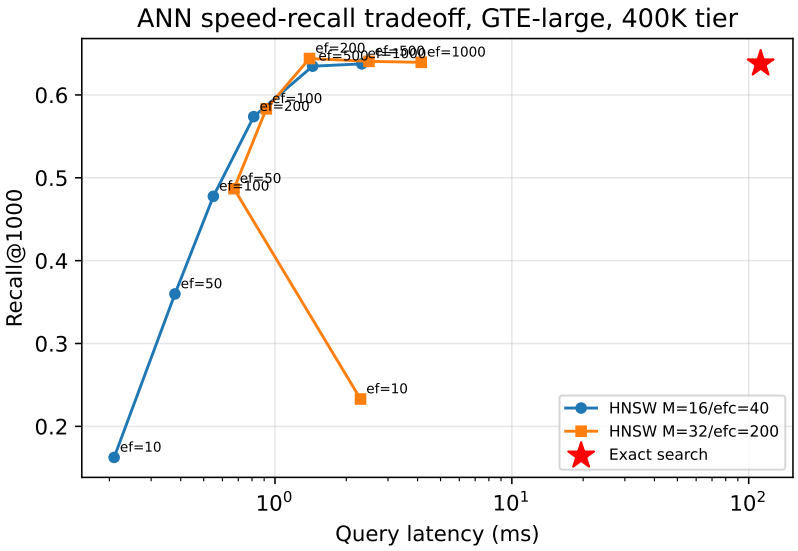


Figure 5.3: ANN speed-recall tradeoff at the ~400K tier (GTE-large): Recall@1,000 vs. query latency across the `ef_search` sweep, both build configurations, against the exact-search reference point.

pared directly. What matters is that both experiments, one on synthetic filler data early in this thesis, one on the real, nested scaling corpus built later, agree on the same qualitative conclusion. Approximate search is a large, low-risk efficiency gain for this retrieval task, with negligible quality cost at a well-chosen operating point.

5.6 Overall Experimental Findings

Table 5.27 summarises the headline result from each experimental phase; Chapter 6 draws these findings together against this thesis's original research questions.

Read honestly and without overclaiming (Section 5.3.5), a final production-independent test, evaluated exclusively on companies production had no opportunity to already know about, substantially closes the single most important claim this thesis had left open: at the full 397,025-company tested scale, Linq-Embed-Mistral recovers 88.5% of independently, blindly verified relevant companies, higher than this thesis's original headline Recall@1,000 figure. This is not, strictly, the symmetric head-to-head comparison against production's own ranking that Section 4.3.2 set out to make, no such comparison is definable for a candidate pool production never ranked at all, but it is the strongest available evidence that this thesis's coverage gains hold up once the pooling bias favouring production is removed by construction.

Mapped against this thesis's three research questions (Chapter 1), these results speak to each directly. RQ1 asked whether semantic retrieval generates accurate candidate sets efficiently and compares favourably to production; the ANN scaling results and

Table 5.27: Final comparison across the pipelines evaluated in this chapter. Each result is reported under the protocol it was actually evaluated against, since not every method was re-evaluated under every protocol (Table 4.1 defines each protocol).

Method	**Protocol**	**Headline metric**	**Interpretation**
BM25	Pseudo-relevance	Recall@1,000 = 0.635	Weakest baseline; limited by vocabulary mismatch
Best single dense model (MiniLM)	Pseudo-relevance	Recall@1,000 = 0.741	Strongest baseline despite smallest model size
3-way hybrid + reranker	Pseudo-relevance	Recall@1,000 = 0.804	Complementary signals close part of the oracle gap
Learned fusion ranker	Pseudo-relevance	Recall@1,000 = 0.823	Best result under the original protocol
Learned fusion ranker (retrained)	Silver standard	NDCG@10: 0.930 → 0.952	Ranking quality holds; feature-importance interpretation reverses
Neural network fusion ranker	Silver standard	NDCG@1,000: 0.286 → 0.360	Broad-list gain over tuned GBDT is statistically significant
Linq-Embed-Mistral	Production-independent	Recall@1,000 = 0.885	Strongest available evidence of genuine coverage gain
HNSW (ANN)	Silver standard, scaling tiers	Within 0.5–2.5% of exact search	Large speed gain (25–78×), no measurable quality cost

the production-independent test both support this, though the one direct, symmetric ranking comparison available favoured production instead (Section 5.3.3), a discrepancy this thesis attributes to pooling bias in that comparison's candidate pool rather than a genuine quality reversal. RQ2 asked whether combining retrieval signals retrieves more relevant companies than any single method alone; the fusion ranker's Recall@1,000 gain over every single-model baseline answers this directly, though which signal it relies on most changes once evaluated against the independent silver standard rather than the original, circular protocol. RQ3 asked whether the original evaluation protocol is itself trustworthy; on its own it is not, production's own results miss more than a quarter of independently verified relevant companies, which is exactly why every result in this section is reported alongside the protocol it was measured under, rather than as a single, unqualified number. Chapter 6 now answers each research question in full against these results.

6 Conclusion

This thesis set out to answer three research questions about semantic company retrieval for ISTARI.AI's search product (Chapter 1). It compared lexical and dense retrieval, hybrid fusion, and a learned ranker trained on this task's own queries against Istari's keyword-based production system (Chapter 5), then discovered that the evaluation protocol used to measure all of this was itself circular, since it defined relevance as whatever production had already retrieved. It built an independent, multi-judge silver standard to test whether the thesis's own conclusions survived that correction, and re-ran the retrieval and scaling experiments against it. The remainder of this chapter revisits each research question in turn and answers it directly against these results.

RQ1: retrieval effectiveness and efficiency. This thesis asked whether embedding-based semantic retrieval, combined with approximate nearest-neighbour search, can generate accurate top-k candidate sets from a large-scale dataset while keeping query latency low, and how its quality compares to Istari's production system. All three dense baselines outperform BM25 at every cut-off tested (Table 5.1), addressing the vocabulary-mismatch problem keyword matching cannot. At the largest tested scale, 397,025 companies, approximate nearest-neighbour search matches exact search's retrieval quality within 0.5–2.5 percentage points while running 25–78× faster, a gap confirmed by paired significance testing rather than asserted from point estimates alone (Section 5.5.2). Tested independently of production's own output, on a pool of companies production had never ranked at all, Linq-Embed-Mistral recovers 88.5% of independently verified relevant companies (Section 5.3.4), while production's own ranking misses 27.3% of that same independently verified set (Section 5.3.3). The answer is yes on both counts, with one caveat. Semantic retrieval generates accurate candidate sets efficiently at the scale tested, and it substantially improves on production's own coverage of independently verified relevant companies. A direct, symmetric head-to-head ranking comparison was also attempted (Section 5.3.3) and found the opposite result, production ahead, but that comparison's candidate pool was itself partly drawn from production's own output, giving production a structural advantage the production-independent test was specifically designed to remove.

RQ2: combining retrieval signals. This thesis asked whether combining multiple, complementary retrieval signals, culminating in a model trained directly on this task's own queries, retrieves more relevant companies than any single retrieval method alone. A 3-way hybrid and reranking pipeline raised Recall@1,000 to 0.804 over the best single dense baseline's 0.741 (Section 5.2.7), and a learned fusion ranker trained directly on this

task's own queries reached 0.823, the strongest result under the original protocol (Section 5.2.8). Retrained and evaluated against the independent silver standard instead, the fusion ranker's ranking quality held and improved (NDCG@10: 0.930 to 0.952), though one of its own interpretive findings, about which baseline signal mattered most, did not survive the same test (Section 5.3.2). The answer is also yes, with an important caveat: combining signals genuinely retrieves more relevant companies than any single method, but which signal the fusion ranker learned to rely on most is less certain than the original, circular evaluation suggested.

RQ3: is the evaluation protocol trustworthy? This thesis asked whether the production-derived evaluation protocol used to answer RQ1 and RQ2 is itself trustworthy, and whether the retrieval gains observed under it hold once tested against an independently constructed, non-circular relevance standard. On its own, the protocol is not trustworthy: it defines relevance as whatever production itself already retrieved, so it can never show a system beats production by construction. This thesis measured exactly how much that circularity was hiding by building an independent, multi-judge silver standard instead (Section 4.3.1). Under that standard, production's own results miss 27.3% of independently verified relevant companies (Section 5.3.3), and the fusion ranker's own feature-importance finding reverses (Section 5.3.2), while its broader ranking quality does not. The production-independent test (Section 5.3.4) goes a step further, testing companies production never ranked at all rather than only re-scoring an already production-pooled candidate set, and finds 88.5% recall there too. Taken together, RQ1 and RQ2's central conclusions, that semantic retrieval improves on production's coverage and that combining signals helps, survive this correction, though not as an unqualified, symmetric head-to-head win, the one direct test of that specific claim favoured production instead (Section 5.3.3). What does not survive fully intact is the original protocol's authority to make that claim on its own; it takes an independent standard, built and validated separately from production, to show it convincingly.

Together, these results make a concrete case that semantic retrieval can improve retrieval coverage while maintaining substantial efficiency advantages over a keyword-based production system, and that trusting a production system's own output as ground truth, when no human-labelled data exists, deserves far more scrutiny than it typically receives. Chapter 7 discusses what remains open, most notably testing these conclusions at Istari's full production scale.

7 Future Work

Several concrete extensions were identified over the course of this thesis but not implemented, either for lack of time or because they depend on results not yet available. They are grouped here by what they would establish.

Finding companies production missed. The single most important claim this thesis had left open was whether its retrieval pipeline finds relevant companies production never surfaced, tested fairly rather than against a candidate pool partly pooled from production's own output. Section 5.3.4 now runs exactly this test, on the random, production-independent 300,000-company sample, and finds strong recall (Linq-Embed-Mistral reaching 0.885 at full scale) on a pool clean of that bias by construction, substantially, if not completely, closing the gap. One smaller piece of this remains. Re-scoring the original baseline embedding models' already-computed rankings against the current, non-circular silver standard, at no additional retrieval or encoding cost, would check whether the original baseline comparison (Section 5.2.1) survives independent verification the same way the fusion ranker's own feature-importance finding was checked and found to partially reverse (Section 5.3.2).

Strengthening the evaluation methodology. Precision figures throughout this thesis are conservative lower bounds rather than exact values, since the silver standard is pooled rather than exhaustive (Section 5.3.5). A more exhaustive LLM ranking pass over an already-pooled candidate set, rather than the smaller actively-sampled subset judged here, would tighten this bound. It would need to keep drawing candidates from that same pooled, multi-source set rather than a single retrieval method's own output, otherwise the single-source circularity the silver standard was built to avoid reappears one layer downstream.

Paired significance testing (Wilcoxon signed-rank and paired bootstrap confidence intervals) has now been added for the three comparisons in this thesis where a directional claim was most load-bearing: tuned vs. untuned fusion ranker, neural network vs. tree ensemble, and ANN vs. exact search. The results were not uniformly confirmatory. The hyperparameter-tuning trade-off did not reach significance on the 19-query leakage-free sample, the neural network's broad-list (NDCG@1,000) gain was confirmed but its top-of-list (NDCG@10) gain was not, and ANN's near-equivalence to exact search was confirmed directly (Sections 5.4.1, 5.5.2). Extending the same testing to the remaining comparisons not yet covered (fine-tuned vs. pretrained reranker, production vs. independent ranking) is a direct next step using the same per-query data and methodology.

The classifier's predicted probabilities are only moderately well calibrated, with a systematic gap in the low-to-middle confidence range (Section 5.3.5). Probability recal-

ibration, for example Platt scaling or isotonic regression, before applying the fixed 0.5 relevance threshold used throughout, is a direct, well-understood fix. Label propagation from judged candidates to their nearest, unjudged neighbours in embedding space (Zhu and Ghahramani 2002) remains a cheaper, complementary way to grow labelled data, identified during the active-learning expansion (Section 4.3.2) but not implemented.

Judging all 200 candidates returned by the joint external review with the annotator at Istari fresh with the current LLM ensemble closed part of this gap (92.5% agreement, kappa 0.196, suppressed by label-distribution skew rather than genuine disagreement, Section 5.3.6), entirely without the author's involvement. A genuinely blind, non-circular human check on this thesis's own silver labels was also obtained a different way. A self-labelling exercise over a stratified 300-candidate sample, blind to the existing silver labels, reached a raw agreement of 78.7% and a kappa of 0.603 (Section 5.3.1). The remaining open item is a second, independent external reviewer at Istari, so that a genuine inter-reviewer kappa becomes possible rather than only a comparison against the author's own labels. A cleaner fix for the tie-breaking conflict of interest specifically would be blind tie-breaking, routing disagreement cases to a reviewer, ideally not the author, without revealing which system's ranking surfaced the candidate or at what position, so the resolution cannot be influenced by which result it will end up scoring.

Further modelling experiments. The neural network fusion ranker (Section 4.4.1) used a random hyperparameter search over a single train/validation split rather than the fully cross-validated search used for the tree ensemble. Bayesian optimisation, mentioned as a candidate strategy but not implemented for lack of a suitable dependency, could search the same space more efficiently and with a proper nested cross-validated estimate of its final performance.

Fine-tuning was applied to the reranker (Section 4.4.2) but not to a bi-encoder embedding model directly, a larger and riskier undertaking given the modest size of the current silver standard, but one with real potential to improve first-stage retrieval quality rather than only reranking.

The boilerplate/platform-summary contamination identified in the underlying corpus (Section 5.2.4) was quantified but not corrected in any result reported here. Using the boilerplate-similarity score as an input feature to a trained model, rather than as a hard filter, would let the model learn how much to trust an imperfect signal instead of requiring it to be reliable enough for a fixed decision rule.

Scaling further. Every experiment in this thesis operates on a corpus of at most 397,025 companies, roughly 2% of Istari's full 20-million-company database. The ANN scaling results (Sections 5.2.3, 5.5.2) are consistent across both a synthetic and a real test at this scale, pointing towards the same favourable trade-off, but neither measures behaviour at the full production scale. Testing whether retrieval quality, ANN's efficiency advantage, and the silver-standard construction pipeline itself all continue to hold at that scale would require a substantially larger data export from Istari than used in this thesis. That is the natural next step towards a production deployment of the methods developed here.

Bibliography

[1] Sophia Althammer, Guido Zuccon, Sebastian Hofstätter, Suzan Verberne, and Allan Hanbury. Annotating data for fine-tuning a neural ranker? current active learning strategies are not better than random selection. In *Proceedings of the Annual International ACM SIGIR Conference on Research and Development in Information Retrieval in the Asia Pacific Region*, 2023. doi: 10.1145/3624918.3625333.

[2] Ron Artstein and Massimo Poesio. Inter-coder agreement for computational linguistics. *Computational Linguistics*, 34(4):555–596, 2008. doi: 10.1162/coli.07-034-R2.

[3] BAAI. bge-reranker-v2-m3. https://huggingface.co/BAAI/bge-reranker-v2-m3, 2024. Model card.

[4] Ziv Bar-Yossef and Maxim Gurevich. Random sampling from a search engine's index. *Journal of the ACM*, 55(5), 2008. doi: 10.1145/1411509.1411514.

[5] Steven Bird, Ewan Klein, and Edward Loper. NLTK: the natural language toolkit, 2009.

[6] Chris Buckley and Ellen M. Voorhees. Retrieval evaluation with incomplete information. In *Proceedings of the 27th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 25–32, 2004. doi: 10.1145/1008992.1009000.

[7] Jianlv Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. BGE m3-embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation, 2024.

[8] Jacob Cohen. A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 20(1):37–46, 1960. doi: 10.1177/001316446002000104.

[9] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Büttcher. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 758–759, 2009.

[10] Thibault Formal, Benjamin Piwowarski, and Stéphane Clinchant. SPLADE: Sparse lexical and expansion model for first stage ranking. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 2288–2292. ACM, 2021. doi: 10.1145/3404835.3463098.

[11] Christopher Gerling. Company2vec: German company embeddings based on corporate websites, 2023.

[12] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2019.

[13] Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6769–6781, 2020.

[14] Omar Khattab and Matei Zaharia. Colbert: Efficient and effective passage search via contextualized late interaction over bert. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 39–48. ACM, 2020. doi: 10.1145/3397271.3401075.

[15] J. Richard Landis and Gary G. Koch. The measurement of observer agreement for categorical data. *Biometrics*, 33(1):159–174, 1977. doi: 10.2307/2529310.

[16] Dawei Li, Bohan Jiang, Liangjie Huang, Alimohammad Beigi, Chengshuai Zhao, Zhen Tan, Amrita Bhattacharjee, Yuxuan Jiang, Canyu Chen, Tianhao Wu, Kai Shu, Lu Cheng, and Huan Liu. From generation to judgment: Opportunities and challenges of LLM-as-a-judge, 2024.

[17] Yuliang Li, Jinfeng Li, Yoshihiko Suhara, AnHai Doan, and Wang-Chiew Tan. Deep entity matching with pre-trained language models, 2020.

[18] Zehan Li, Xin Zhang, Yanzhao Zhang, Dingkun Long, Pengjun Xie, and Meishan Zhang. Towards general text embeddings with multi-stage contrastive learning, 2023.

[19] Linq AI Research. Linq-embed-mistral technical report, 2024.

[20] Tie-Yan Liu. Learning to rank for information retrieval. *Foundations and Trends® in Information Retrieval*, 3(3):225–331, 2009. doi: 10.1561/1500000016.

[21] Xiaolu Lu, Alistair Moffat, and J. Shane Culpepper. The effect of pooling and evaluation depth on IR metrics. *Information Retrieval*, 2016. doi: 10.1007/s10791-016-9282-6.

[22] Yu A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2020. doi: 10.1109/TPAMI.2018.2889473.

[23] Rui Meng, Ye Liu, Shafiq Rayhan Joty, Caiming Xiong, Yingbo Zhou, and Semih Yavuz. SFR-Embedding-Mistral: Enhance text retrieval with transfer learning. https://blog.salesforceairesearch.com/sfr-embedded-mistral/, 2024. Salesforce AI Research Blog.

[24] Mixedbread AI. mxbai-rerank-large-v1. https://huggingface.co/mixedbread-ai/mxbai-rerank-large-v1, 2024. Model card.

[25] Sidharth Mudgal, Han Li, Theodoros Rekatsinas, AnHai Doan, Youngchoon Park, Ganesh Krishnan, Rohit Deep, Esteban Arcaute, and Vijay Raghavendra. Deep learning for entity matching: A design space exploration. In *Proceedings of the 2018 International Conference on Management of Data*, pages 19–34. ACM, 2018. doi: 10.1145/3183713.3196926.

[26] Niklas Muennighoff, Nouamane Tazi, Loic Magne, and Nils Reimers. MTEB: Massive text embedding benchmark. In *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*, pages 2014–2037, 2023. doi: 10.18653/v1/2023.eacl-main.148.

[27] Rodrigo Nogueira and Kyunghyun Cho. Passage re-ranking with BERT, 2019.

[28] Rodrigo Nogueira, Wei Yang, Jimmy Lin, and Kyunghyun Cho. Document expansion by query prediction, 2019.

[29] Zach Nussbaum, John X. Morris, Brandon Duderstadt, and Andriy Mulyar. Nomic embed: Training a reproducible long context text embedder, 2024.

[30] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[31] Ralph Peeters, Aaron Steiner, and Christian Bizer. Entity matching using large language models. In *Proceedings of the 28th International Conference on Extending Database Technology (EDBT)*, 2025.

[32] Md Mustafizur Rahman, Mucahid Kutlu, and Matthew Lease. Efficient test collection construction via active learning. In *Proceedings of the 2020 ACM SIGIR International Conference on Theory of Information Retrieval*, pages 177–184, 2020. doi: 10.1145/3409256.3409837.

[33] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, pages 3982–3992, 2019.

[34] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009. doi: 10.1561/1500000019.

[35] Gerard Salton and Christopher Buckley. Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, 24(5):513–523, 1988. doi: 10.1016/0306-4573(88)90021-0.

[36] Keshav Santhanam, Omar Khattab, Christopher Potts, and Matei Zaharia. PLAID: An efficient engine for late interaction retrieval. In *Proceedings of the 31st ACM International Conference on Information & Knowledge Management*, pages 1747–1756. ACM, 2022a. doi: 10.1145/3511808.3557325.

[37] Keshav Santhanam, Omar Khattab, Jon Saad-Falcon, Christopher Potts, and Matei Zaharia. Colbertv2: Effective and efficient retrieval via lightweight late interaction. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 3715–3734. Association for Computational Linguistics, 2022b. doi: 10.18653/v1/2022.naacl-main.272.

[38] Burr Settles. Active learning literature survey. Technical Report Computer Sciences Technical Report 1648, University of Wisconsin-Madison, 2009.

[39] Mark D. Smucker, James Allan, and Ben Carterette. A comparison of statistical significance tests for information retrieval evaluation. In *Proceedings of the Sixteenth ACM Conference on Information and Knowledge Management*, pages 623–632, 2007. doi: 10.1145/1321440.1321528.

[40] Rikiya Takehi, Ellen M. Voorhees, Tetsuya Sakai, and Ian Soboroff. LLM-assisted relevance assessments: When should we ask LLMs for help?, 2024.

[41] Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021.

[42] Paul Thomas, Seth Spielman, Nick Craswell, and Bhaskar Mitra. Large language models can accurately predict searcher preferences, 2024.

[43] Ellen M Voorhees. The philosophy of information retrieval evaluation. In *Evaluation of Cross-Language Information Retrieval Systems: Second Workshop of the Cross-Language Evaluation Forum, CLEF 2001, Revised Papers*, volume 2406 of *Lecture Notes in Computer Science*, pages 355–370. Springer, 2002. doi: 10.1007/3-540-45691-0_34.

[44] Liang Wang, Nan Yang, Xiaolong Huang, Binxing Jiao, Linjun Yang, Daxin Jiang, Rangan Majumder, and Furu Wei. Simlm: Pre-training with representation bottleneck for dense passage retrieval, 2022.

[45] Liang Wang, Nan Yang, Xiaolong Huang, Linjun Yang, Rangan Majumder, and Furu Wei. Improving text embeddings with large language models, 2024.

[46] Yu Xia, Subhojyoti Mukherjee, Zhouhang Xie, Junda Wu, Xintong Li, Ryan Aponte, Hanjia Lyu, Joe Barrow, Hongjie Chen, Franck Dernoncourt, Branislav Kveton, Tong Yu, Ruiyi Zhang, Jiuxiang Gu, Nesreen K. Ahmed, Yu Wang,

Xiang Chen, Hanieh Deilamsalehy, Sungchul Kim, Zhengmian Hu, Yue Zhao, Nedim Lipka, Seunghyun Yoon, Ting-Hao Huang, Zichao Wang, Puneet Mathur, Soumyabrata Pal, Koyel Mukherjee, Zhehao Zhang, Namyong Park, Thien Huu Nguyen, Jiebo Luo, Ryan A. Rossi, and Julian McAuley. From selection to generation: A survey of LLM-based active learning, 2025.

[47] Shitao Xiao, Zheng Liu, Peitian Zhang, Niklas Muennighoff, Defu Lian, and Jian-Yun Nie. C-pack: Packed resources for general chinese embeddings. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 641–649. ACM, 2024. doi: 10.1145/3626772.3657878.

[48] Xiaojin Zhu and Zoubin Ghahramani. Learning from labeled and unlabeled data with label propagation. Technical Report CMU-CALD-02-107, Carnegie Mellon University, Pittsburgh, PA, 2002.

[49] Justin Zobel. How reliable are the results of large-scale information retrieval experiments? In *Proceedings of the 21st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 307–314. ACM, 1998. doi: 10.1145/290941.291014.

Code Availability

The entire code repository is available at https://github.com/marmeepandya/masters-thesis.

Company-level data from Istari's Global Organisation Index (GOI), the evaluation queries, and the production search results used as a comparison baseline are proprietary to ISTARI.AI and are not included in either location. Intermediate result files that do not expose raw company data, including relevance labels and evaluation metrics, are included where possible, so that the analysis can be reproduced conditional on independent access to the underlying GOI data.

Ehrenwörtliche Erklärung

Ich versichere, dass ich die beiliegende Bachelor-, Master-, Seminar-, oder Projektarbeit ohne Hilfe Dritter und ohne Benutzung anderer als der angegebenen Quellen und in der untenstehenden Tabelle angegebenen Hilfsmittel angefertigt und die den benutzten Quellen wörtlich oder inhaltlich entnommenen Stellen als solche kenntlich gemacht habe. Diese Arbeit hat in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen. Ich bin mir bewusst, dass eine falsche Erklärung rechtliche Folgen haben wird.

Declaration of Used AI Tools

Tool	**Purpose**	**Where?**	**Useful?**
Claude Sonnet 5 (Claude Code)	Drafting, editing, and restructuring thesis text; proofreading; LaTeX formatting	Throughout	Yes
ChatGPT (web)	General writing and research assistance	Throughout	Yes
Perplexity (web)	Web research and literature lookup	Throughout	Yes
GPT-4o-mini (API)	LLM relevance judge, original judge pair	Ch. 4–5	Yes
Claude Haiku 4.5 (API)	LLM relevance judge, original judge pair	Ch. 4–5	Yes
GPT-5.4 (API)	LLM relevance judge, upgraded pair (bronze labels, active learning)	Ch. 4–5	Yes
Claude Sonnet 5 (API)	LLM relevance judge, upgraded pair (active learning, production-independent test)	Ch. 4–5	Yes
Gemini 3.6 Flash (API)	LLM relevance judge, paired with Claude Sonnet 5 (production-independent test, external review)	Ch. 4–5	Yes

Unterschrift
Marmee Pandya



Mannheim, den September 15, 2026